The Iby&Aladar Fleischman Faculty of Engineering
Tel Aviv University

Digital VLSI Design

Lecture 8
Clock Tree Synthesis

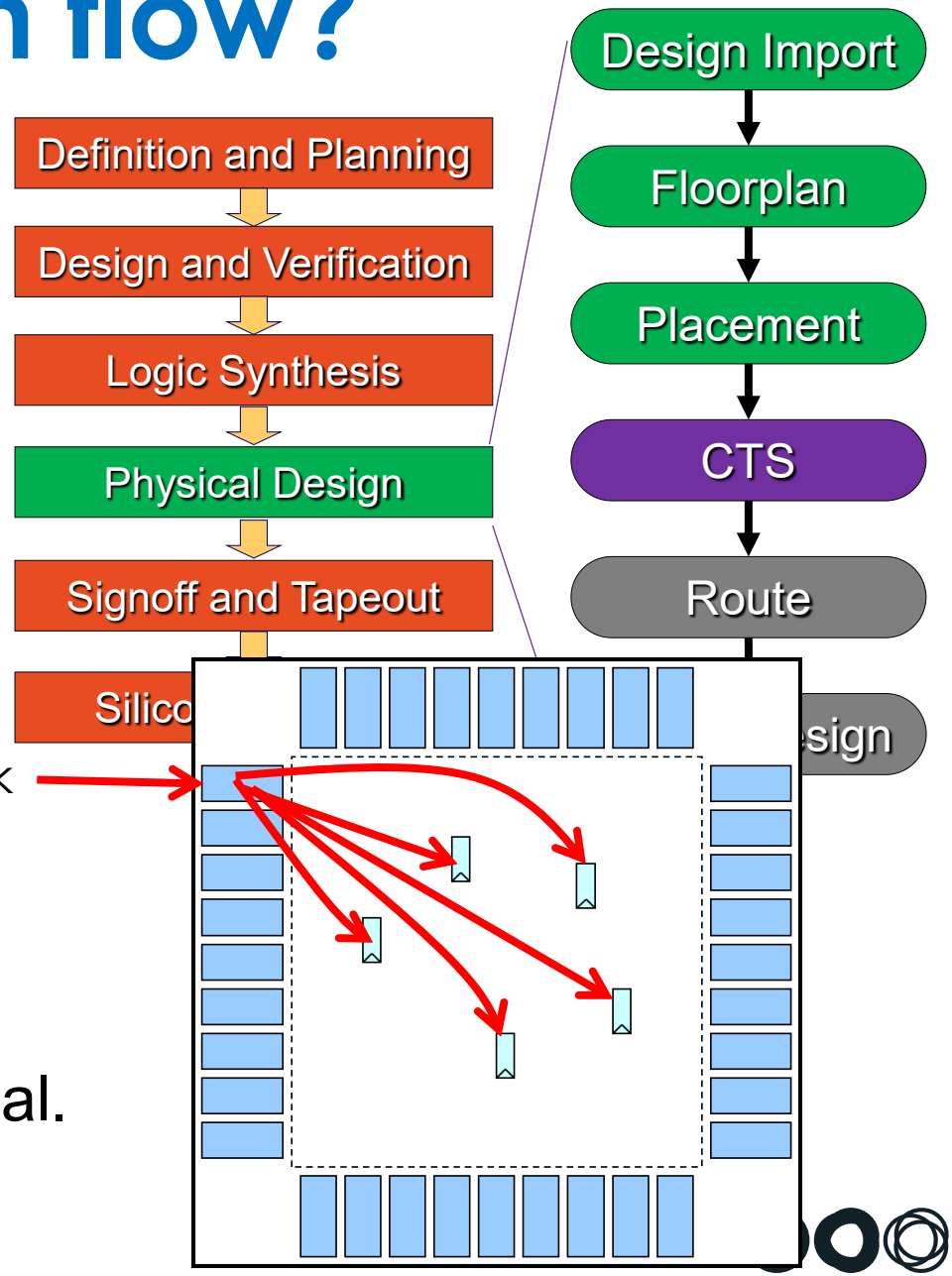
Semester B, 2026

Lecturer: Zvika Webb

15 June 2026

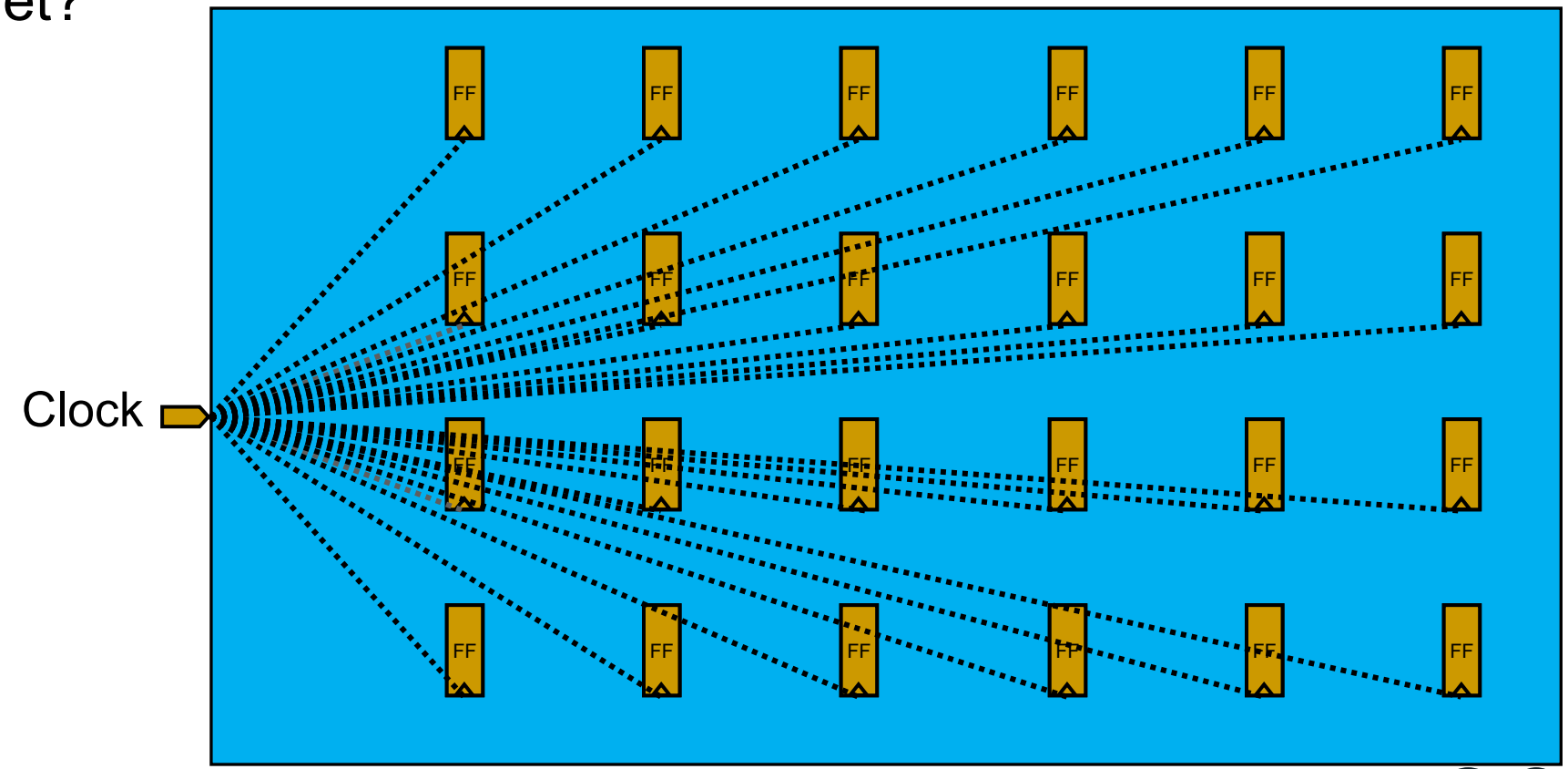
Where are we in the design flow?

- **We have:**
  - Synthesized our design into a technology mapped gate level netlist
  - Designed a floorplan for physical implementation.
  - And provided a location for each and every gate.
- **During all stages:**
  - We analyzed timing constraints and optimized the design according to these constraints.
- **However…**
  - Until now, we have assumed an ideal clock.
  - Now we have <u>all sequential elements placed</u>, so we have to provide them with a ***real*** clock signal.



Trivial Approach?

- **Question:**
  - Why not just route the clock net to all sequential elements, just like any other net?
- **Answer…**
  - Timing
  - Power
  - Area
  - Signal Integrity
  - etc…



Lecture Outline

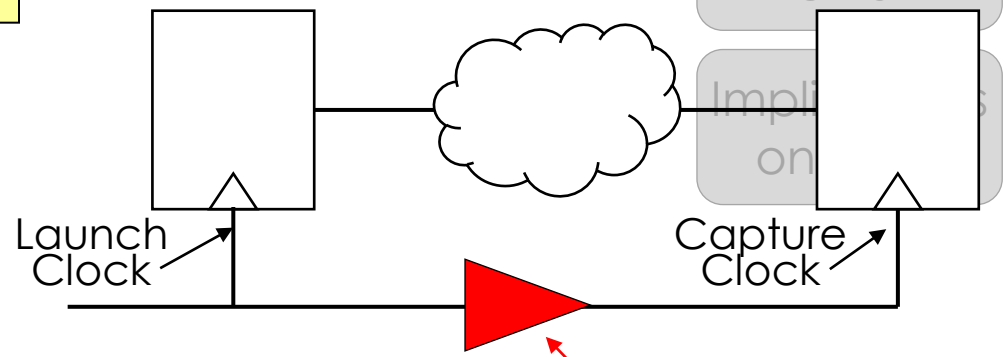
- **Implications of Clocking**
- **Clock Distribution**
- **Clock Tree Synthesis in EDA**
- **Additional Subjects:**
  - Clock Generation
  - Clock Domain Crossing

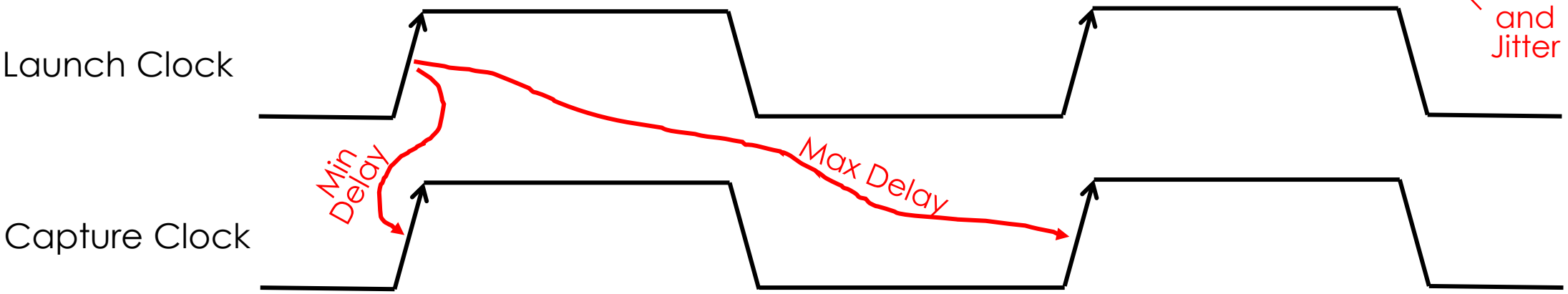
Implications of Clocking

Timing, power, area, signal integrity

Implications on Timing

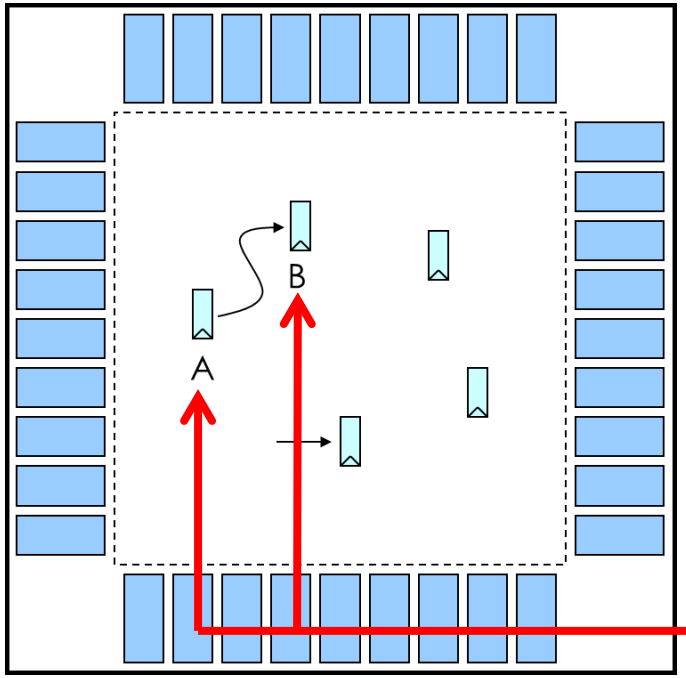
- **Let's remember our famous timing constraints:**
  - Max Delay: $T + \delta_{skew} > t_{CQ} + t_{logic} + t_{setup} + \delta_{margin}$
  - Min Delay: $t_{CQ} + t_{logic} - \delta_{margin} > t_{hold} + \delta_{skew}$

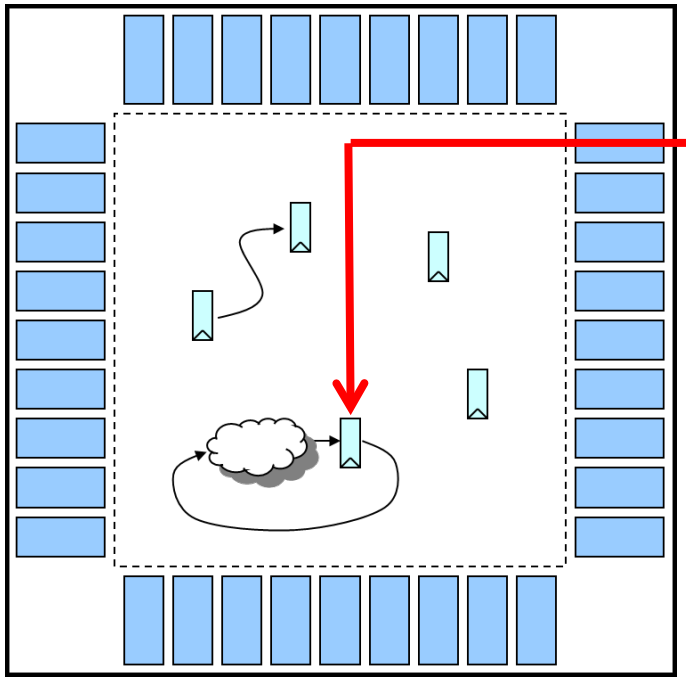




Clock Parameters

- **Skew**
  - Difference in clock arrival time at two different registers.
- **Jitter**
  - Difference in clock period between different cycles.
- **Slew**
  - Transition (t_{rise}/t_{fall}) of clock signal.
- **Insertion Delay**
  - Delay from clock source until registers.



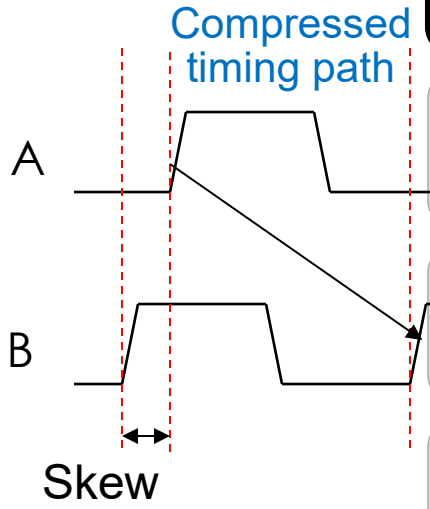


Implications on Timing

Implications on Power

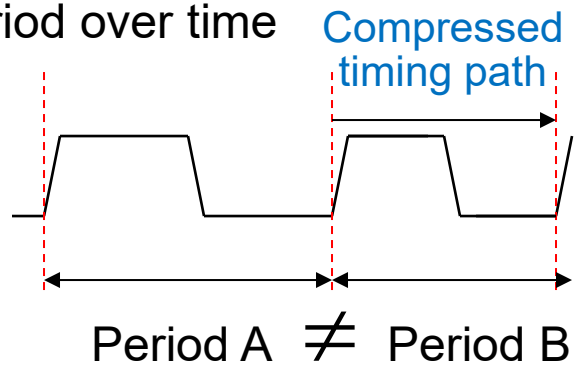
Implications on SI

Implications on Area



Clock Skew
Difference in clock arrival time at two spatially distinct points

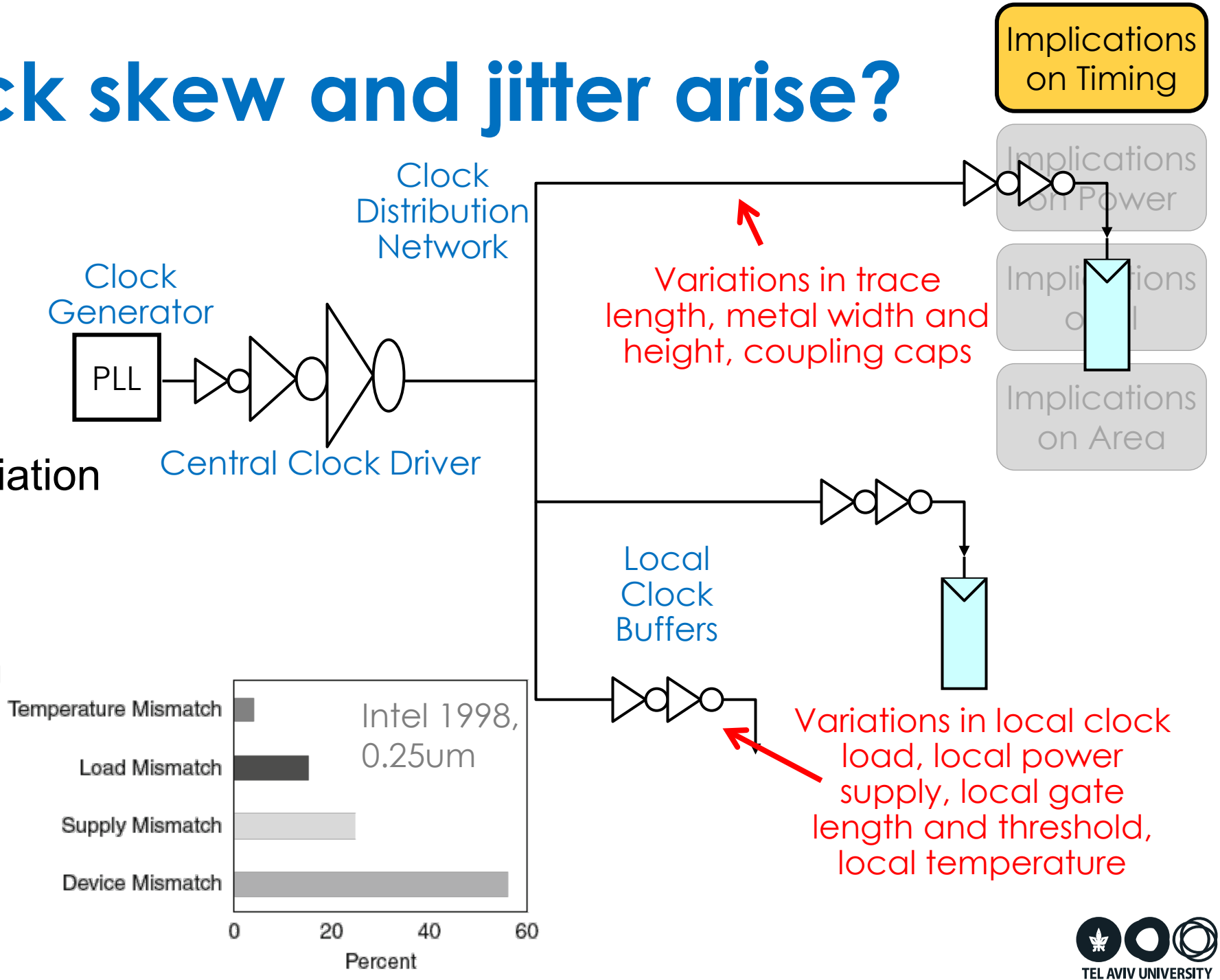
Clock Jitter
Difference in clock period over time



TEL AVIV UNIVERSITY

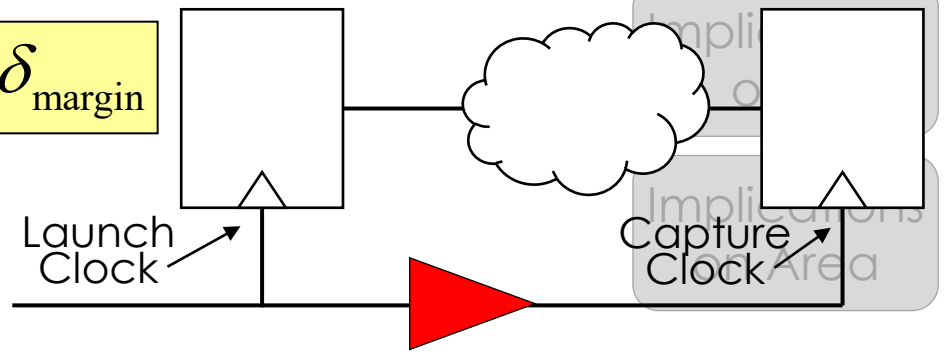
How do clock skew and jitter arise?

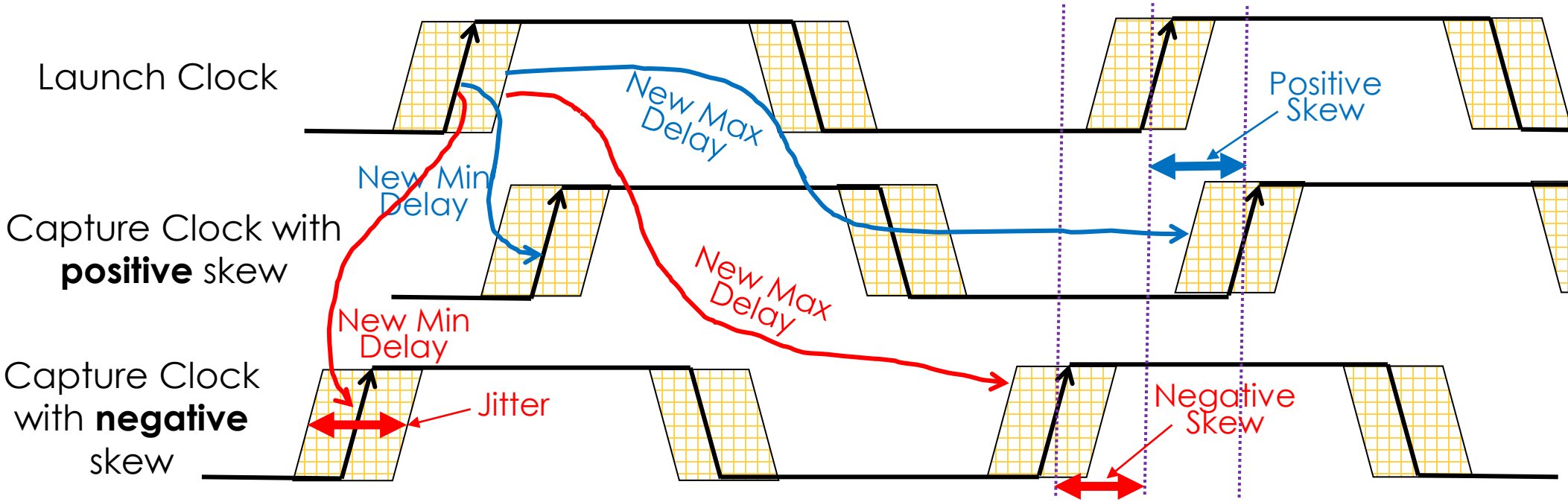
- **Clock Generation**
- **Distribution network**
  - Number of buffers
  - Device Variation
  - Wire length and variation
  - Coupling
  - Load
- **Environment Variation**
  - Temperature
  - Power Supply



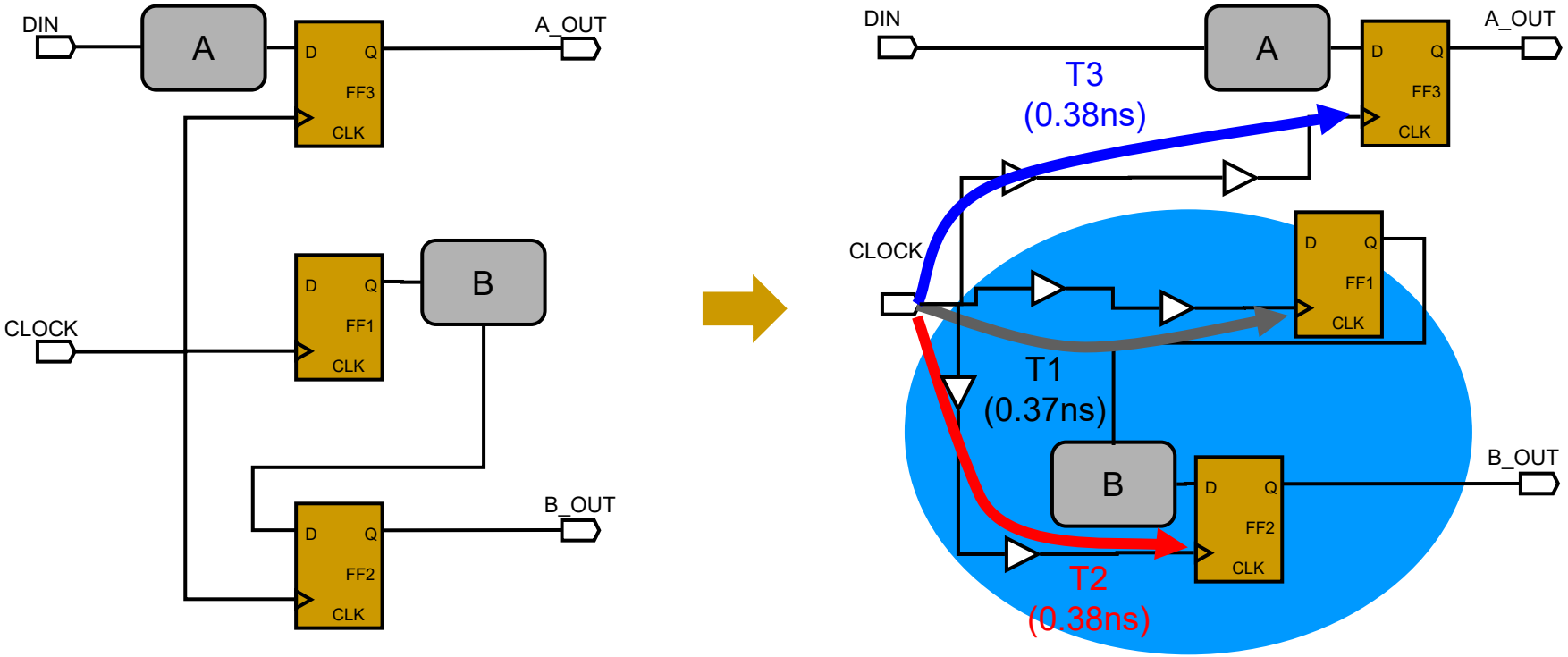
Implications on Timing

- **So skew and jitter eat away at our timing margins:**
  - Max Delay: $T + \delta_{skew} - 2\delta_{jitter} > t_{CQ} + t_{logic} + t_{setup} + \delta_{margin}$
  - Min Delay: $t_{CQ} + t_{logic} - \delta_{margin} > t_{hold} + \delta_{skew}$

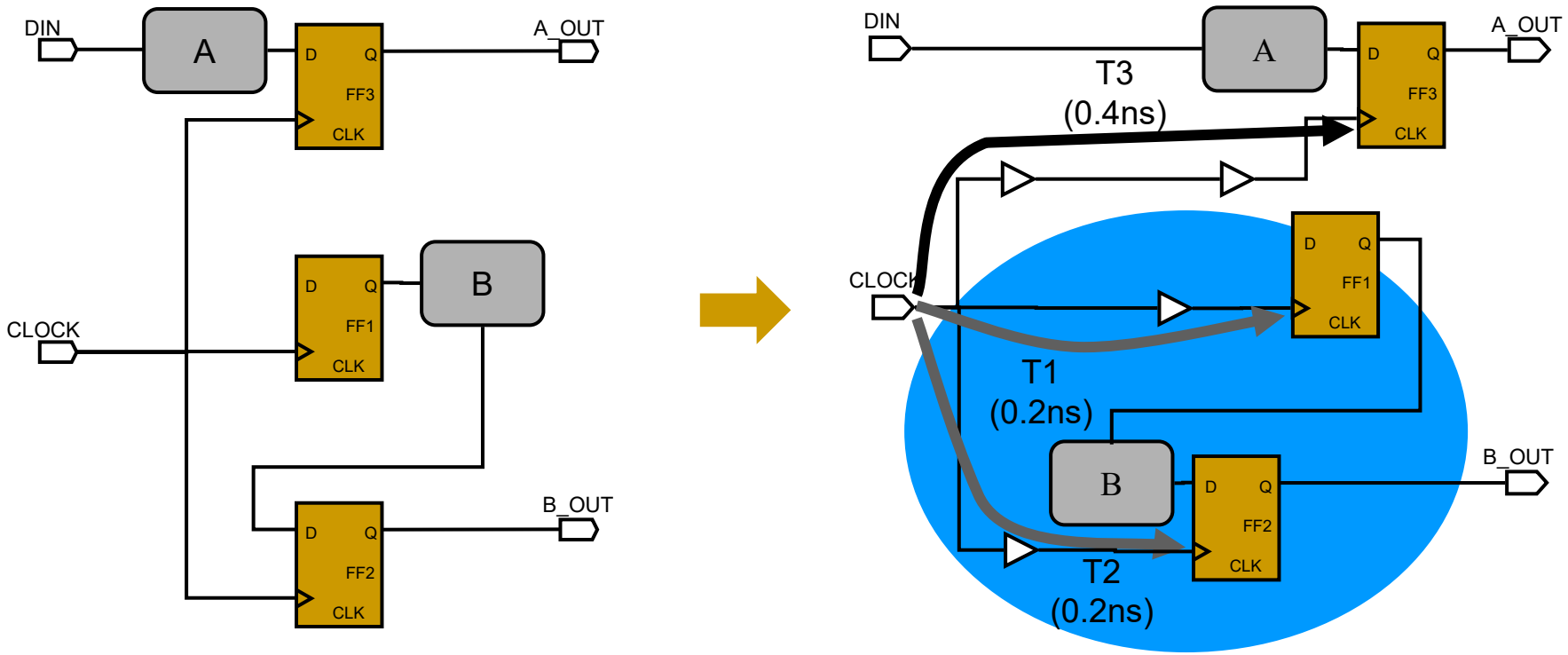




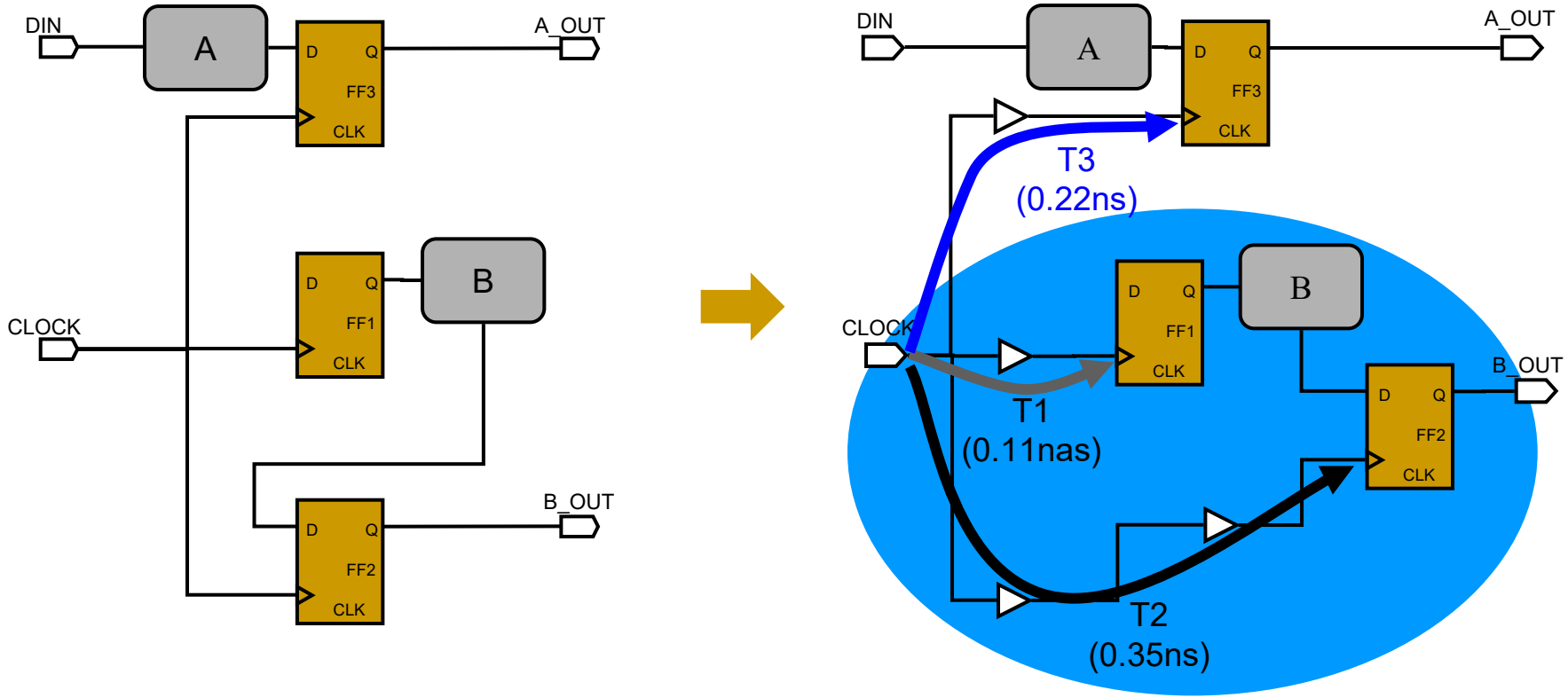
Clock Skew Types: Global



Clock Skew Types: Local



Clock Skew Types: Useful

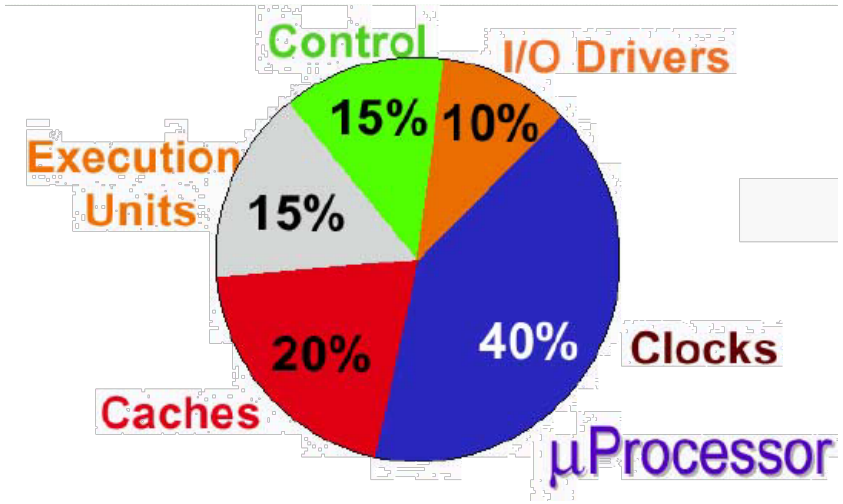


Implications on Power

- **Let's remember how to calculate dynamic power:**

$$P_{dyn} = f \cdot C_{eff} \cdot V_{DD}^2 \qquad C_{eff} = \alpha \cdot C_{total} = \alpha_{clock} \cdot C_{clock} + \alpha_{others} \cdot C_{others}$$

  - The activity factor (α) of the clock network is 100%!
- **The clock capacitance consists of:**
  - Clock generation (i.e., PLL, clock dividers, etc.)
  - Clock elements (buffers, muxes, clock gates)
  - Clock wires
  - Clock load of sequential elements
- **Clock networks are huge**
  - And therefore, the clock is responsible for a large percentage of the total chip power.



Possible power partitioning of a microprocessor (*not general!!!*)

Implications on Signal Integrity

Signal Integrity is an obvious requirement for the clock network:

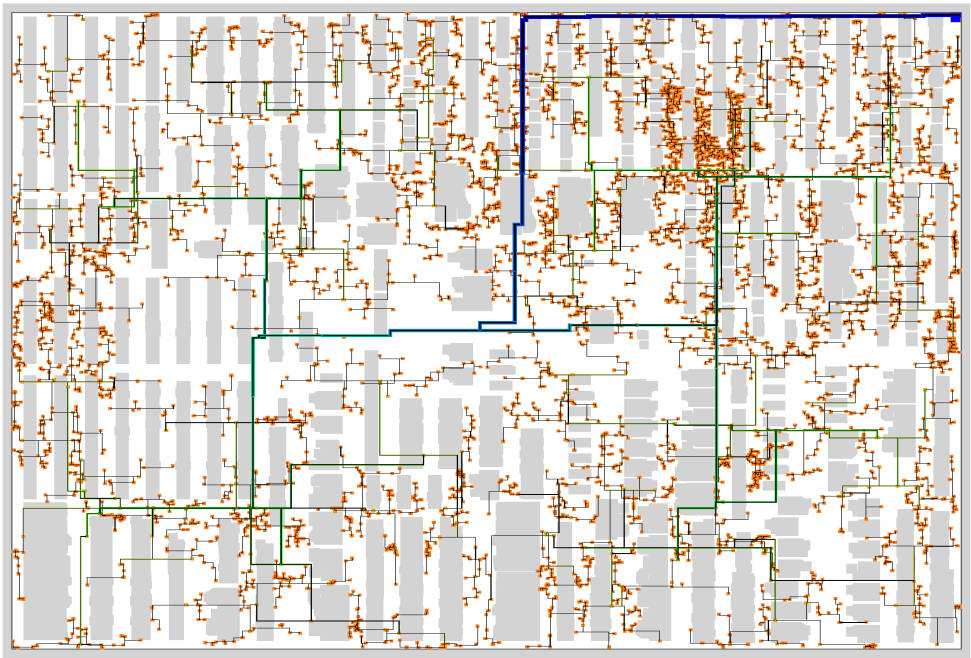
- **Noise on the clock network can cause:**
  - In the worst case, additional clock edges
  - Lower coupling can still slow down or speed up clock propagation
  - Irregular clock edges can impede register operation
- **Slow clock transitions (slew rate):**
  - Susceptibility to noise (weak driver)
  - Poor register functionality (worse t_{cq}, t_{setup}, t_{hold})
- **Too fast clock transitions**
  - Overdesign → power, area, etc.
  - Bigger aggressor to other signals
- **Unbalanced drivers lead to increased skew.**

Best practice:
Keep t_{rise} and t_{fall} between 10-20% of clock period (e.g., 100-200ps @1GHz)

Implications on Area

- **To reiterate, clock networks consist of:**
  - Clock generators
  - Clock elements
  - Clock wires
- **All of these consume area**
  - Clock generators (e.g., PLL) can be very large
  - Clock buffers are distributed all over the place
  - Clock wires consume a lot of routing resources
- **Routing resources are most vital**
  - Require low RC (for transition and power)
    - Benefit of using high, wide metals
  - Need to connect to every clock element (FF)
    - Distribution all over the chip
    - Need Via stack to go down from high metals

For example: **Intel Itanium**
4% of M4/M5 used for clock routing



Source: Universitat Bonn

Clock Distribution

So how do we build a clock tree?

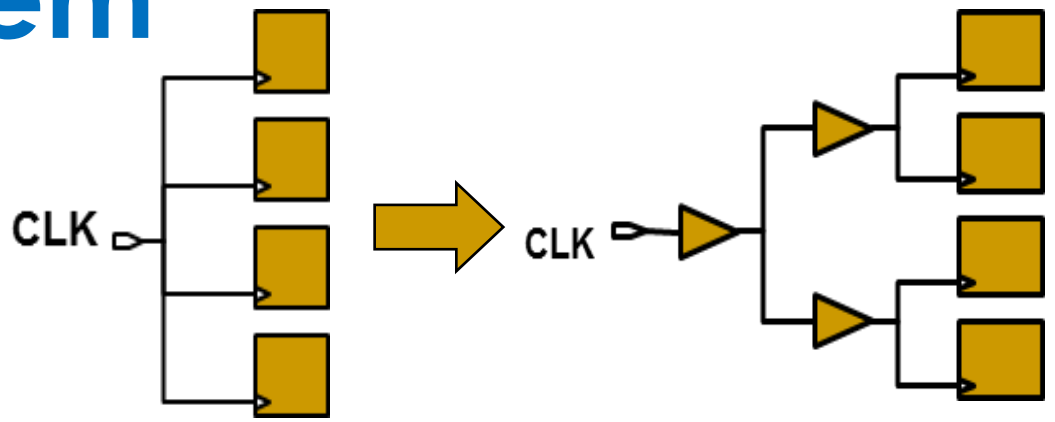
The Clock Routing Problem

Given a source and *n* sinks:

- **Connect all sinks to the source by an interconnect network so as to minimize:**
  - Clock Skew = $\max_{i,j} |t_i - t_j|$
  - Delay = $\max_i (t_i)$
  - Total wirelength
  - Noise and coupling effect

The Challenge:

- **Synchronize millions (billions) of separate elements**
  - Within a time scale on order of ~ 10 ps
  - At distances spanning 2-4 cm
    - Ratio of synchronizing distance to element size on order of 10^5
  - Reference: light travels <1 cm in 10 ps

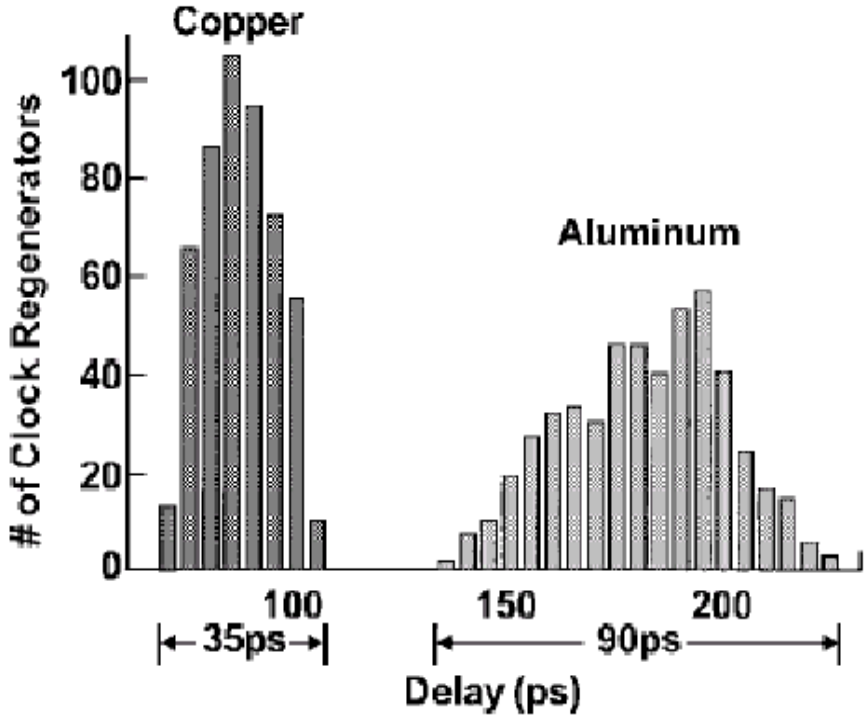


Clock Tree goals:
1) Minimize Skew
2) Meet target insertion delay (min/max)

Clock Tree constraints:
1) Maximum Transition
2) Maximum load cap
3) Maximum Fanout
4) Maximum Buffer Levels

Technology Trends

- **Timing**
  - Higher clock frequency → Lower skew
  - Higher clock frequency → Faster transitions
  - Jitter – PLL's get better with CMOS scaling
  - but other sources of noise increase
    - Power supply noise more important
    - Switching-dependent temperature gradients



- **New Interconnect Materials**
  - Copper Interconnect → Lower RC → Better slew and potential skew
  - Low-k dielectrics → Lower clock power, better latency/skew/slew rates

- **Power**
  - Heavily pipelined design → more registers → more capacitive load for clock
  - Larger chips → more wire-length needed to cover the entire die
  - Complexity → more functionality and devices → more clocked elements
  - Dynamic logic → more clocked elements

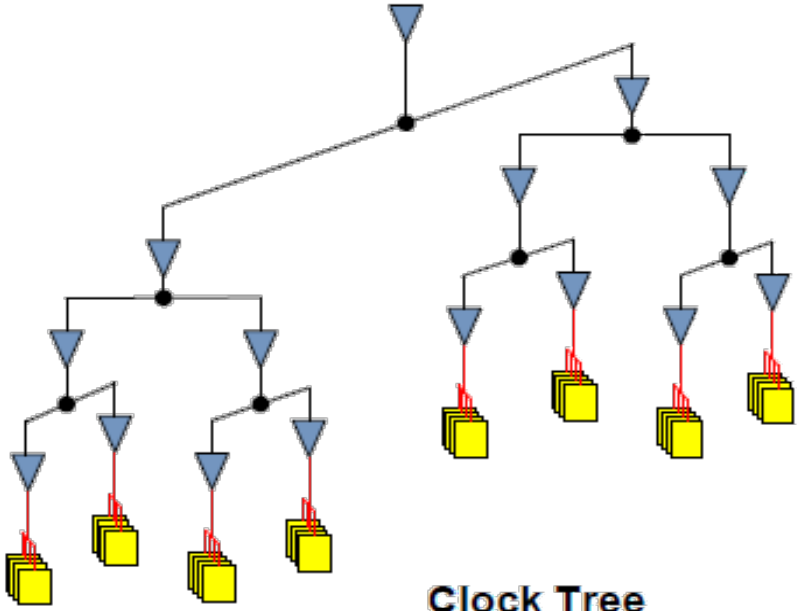
Approaches to Clock Synthesis

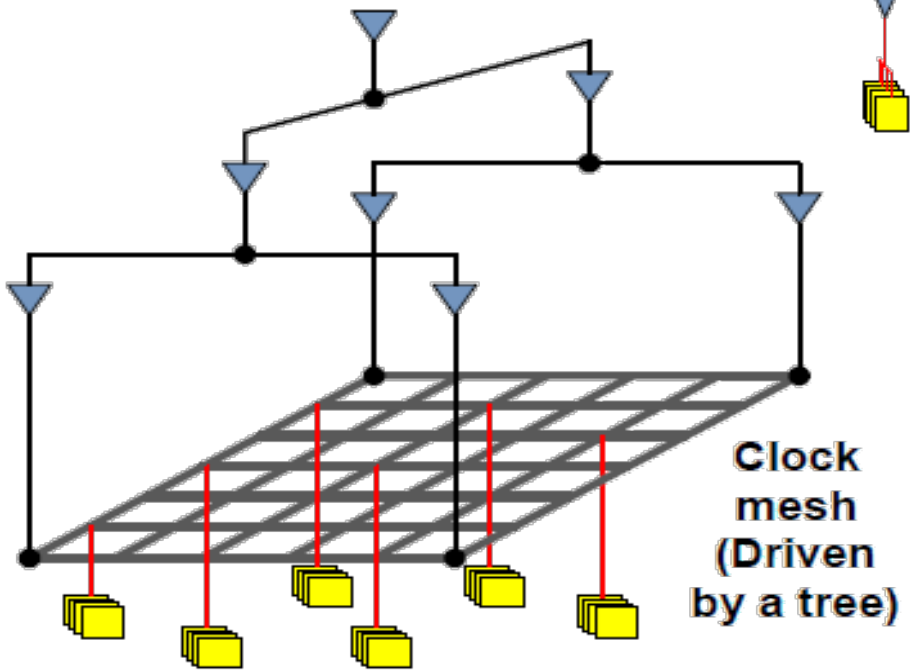
Clock Tree

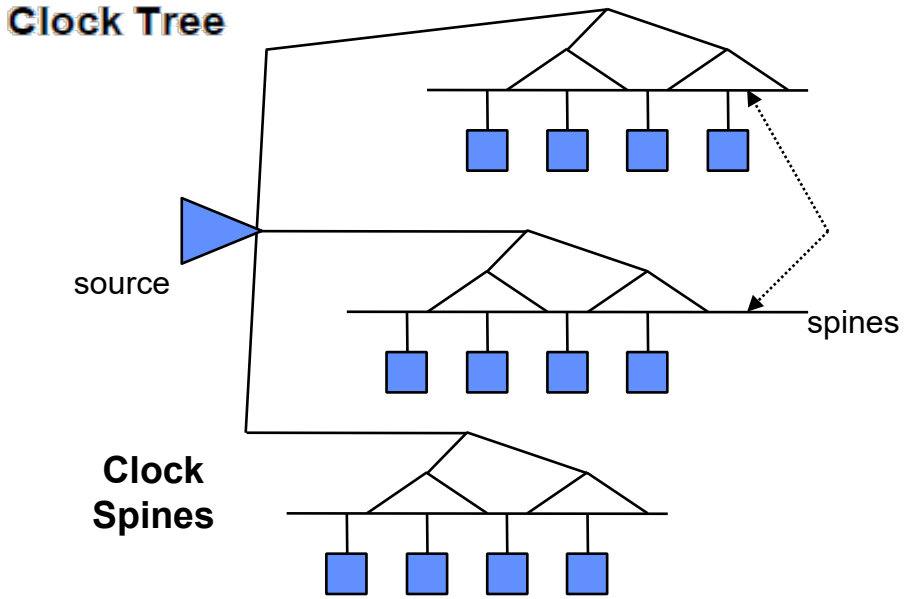
Clock Grid

Clock Spine

- **Broad Classification:**
  - Clock Tree
  - Clock Mesh (Grid)
  - Clock Spines





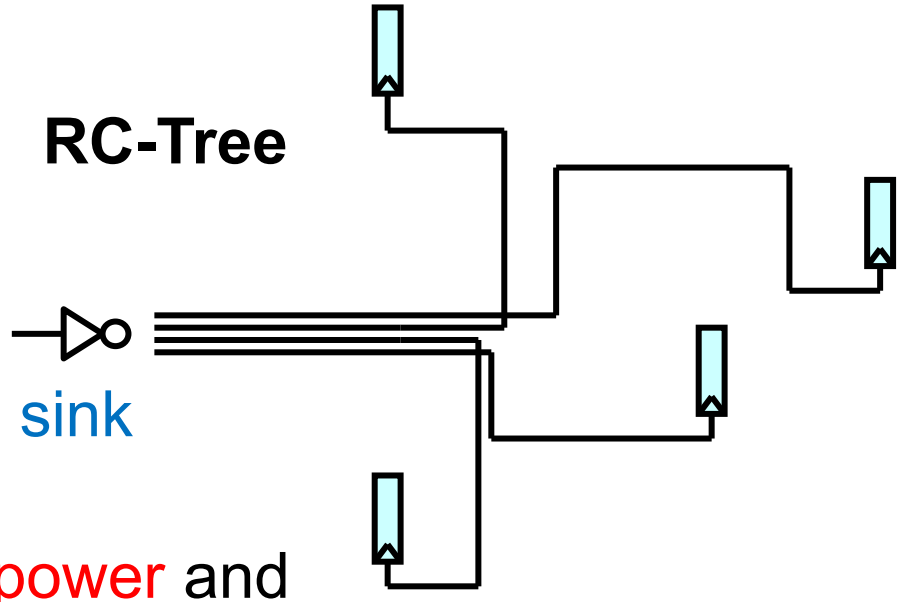


Clock Trees

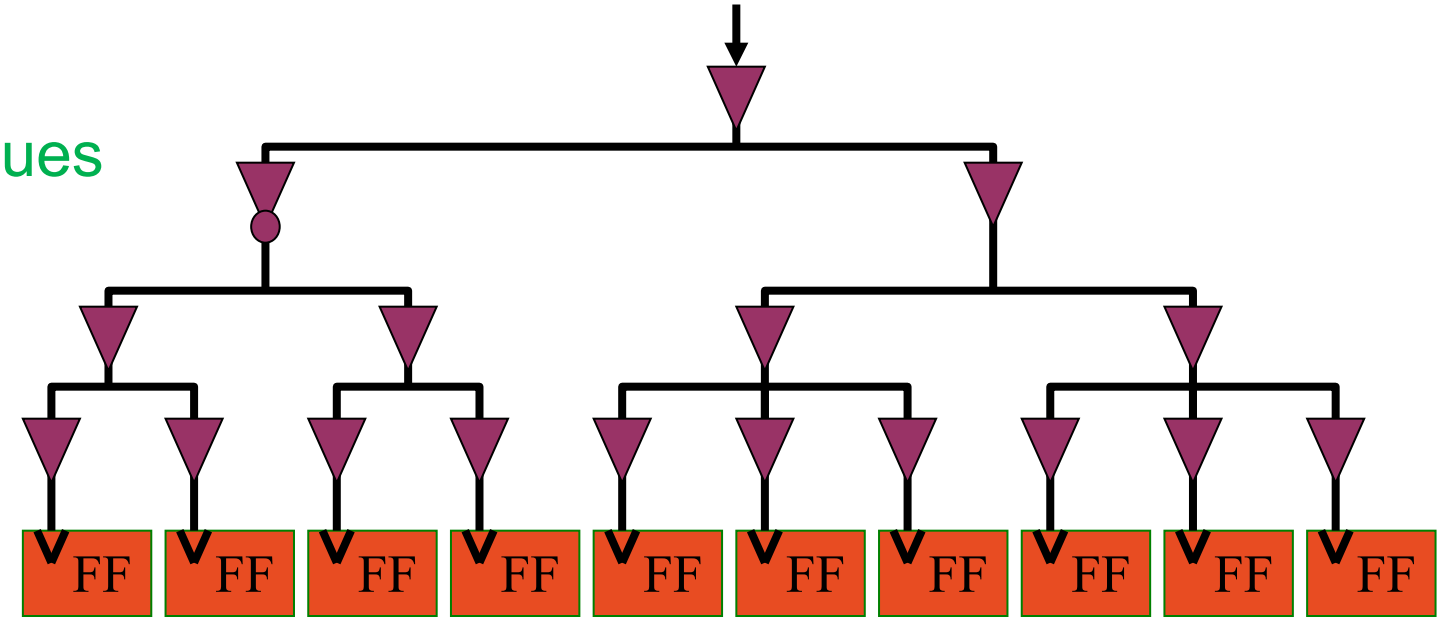
Clock Tree

Clock Grid

Clock Spine



- **Naïve approach:**
  - Route an individual clock net to each sink and balance the RC-delay
  - However, this would burn excessive power and the large RC of each net would cause signal integrity issues.
- **Instead use a buffered tree**
  - Short nets mean lower RC values
  - Buffers restore the signal for better slew rates
  - Lower total insertion delay
  - Less total switching capacitance



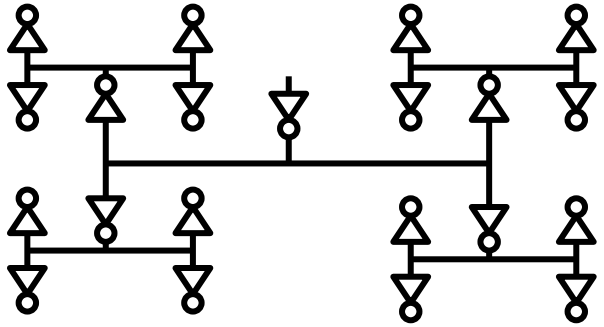
Building an actual Clock Tree

Clock Tree

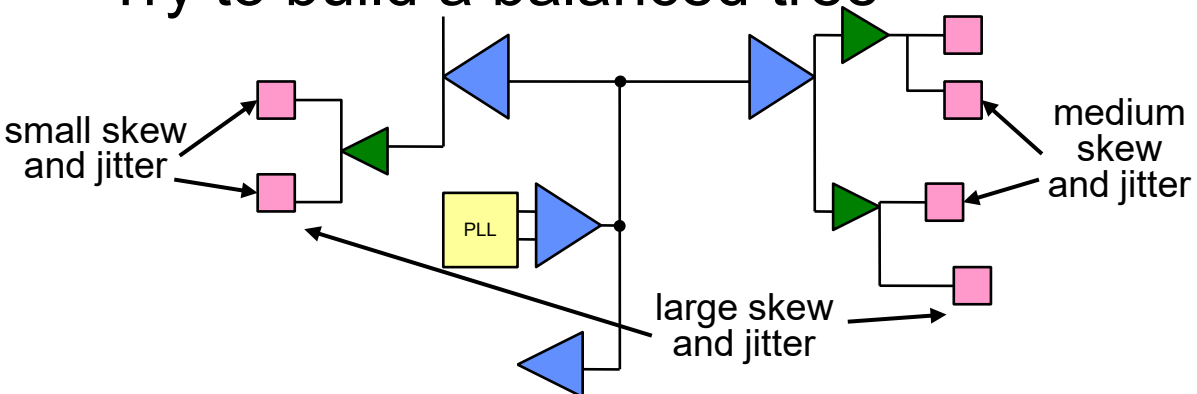
Clock Grid

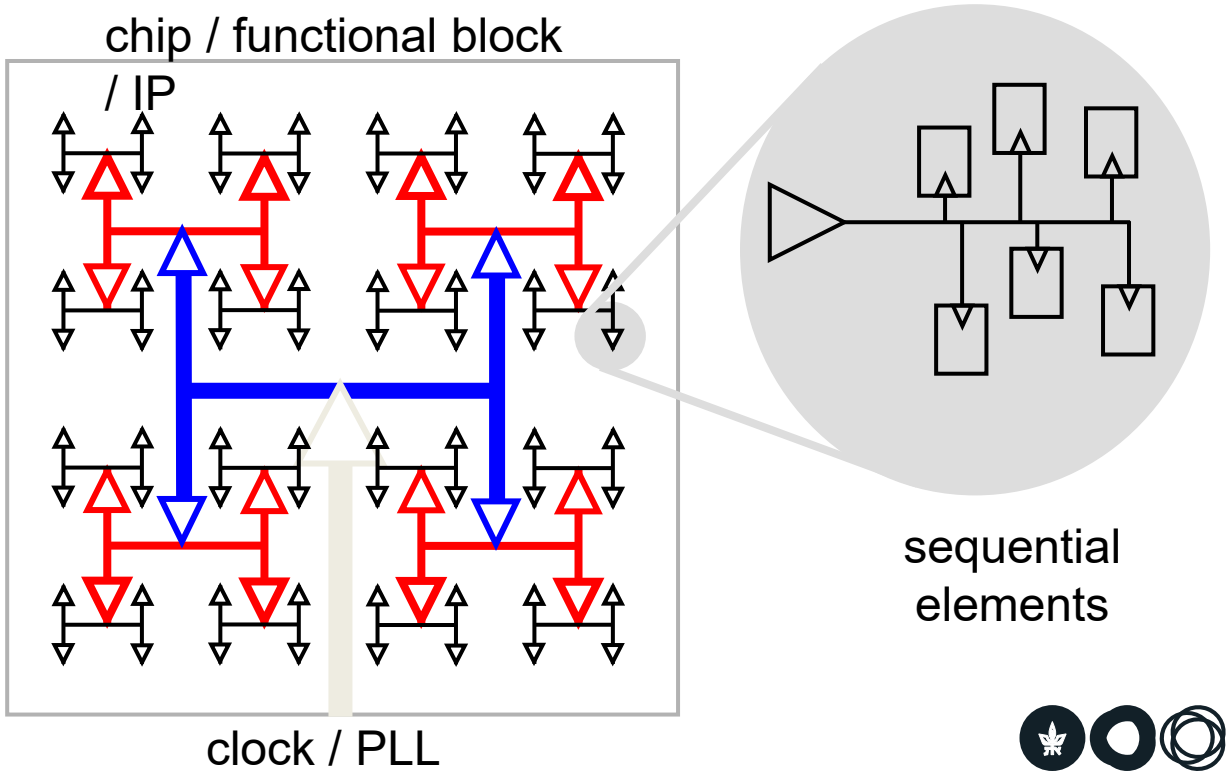
Clock Spine

- **Perfectly balanced approach: H-Tree**
  - One large central driver
  - Recursive H-style structure to match wire-lengths
  - Halve wire width at branching points to reduce reflections



- **More realistic:**
  - Tapered H-Tree, but still hard to do.
- **Standard CTS approach:**
  - Flip flops aren't distributed evenly.
  - Try to build a balanced tree





TEL AVIV UNIVERSITY

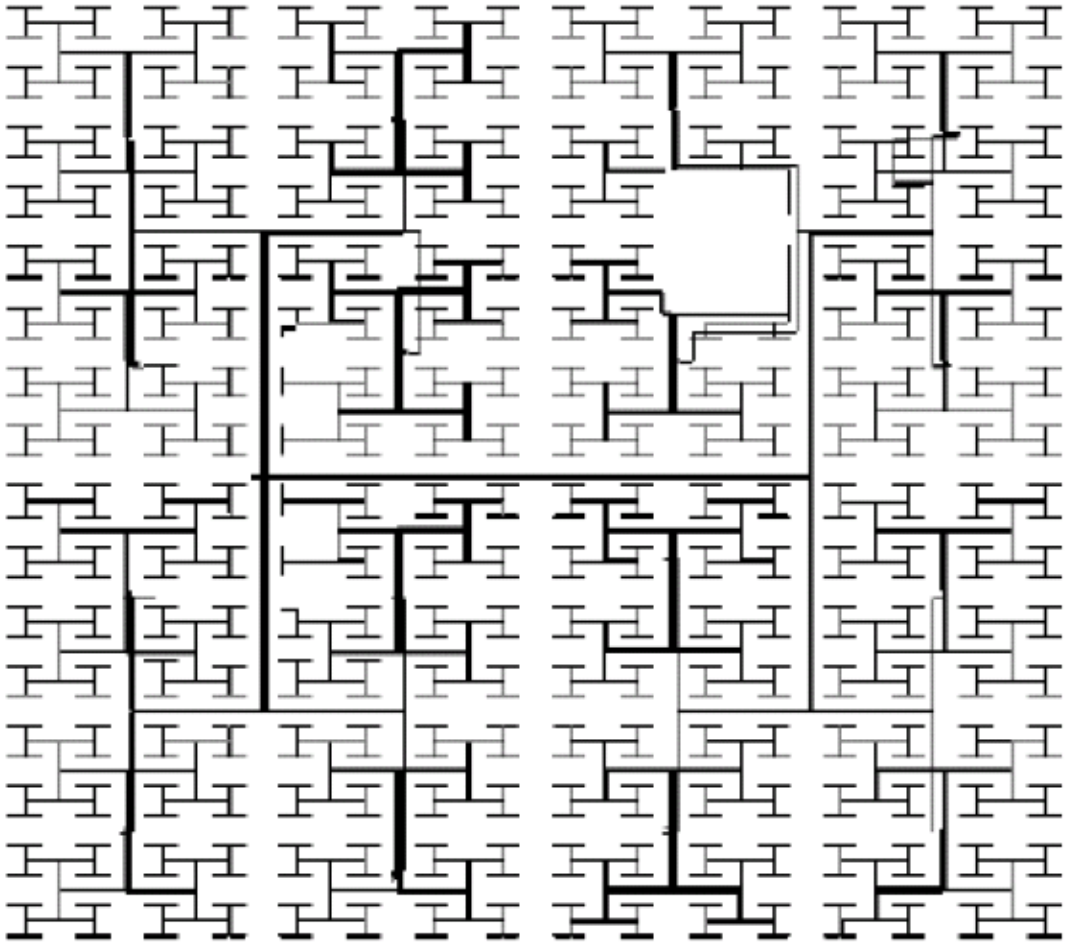
Industrial H-Tree Examples

Clock Tree

Clock Grid

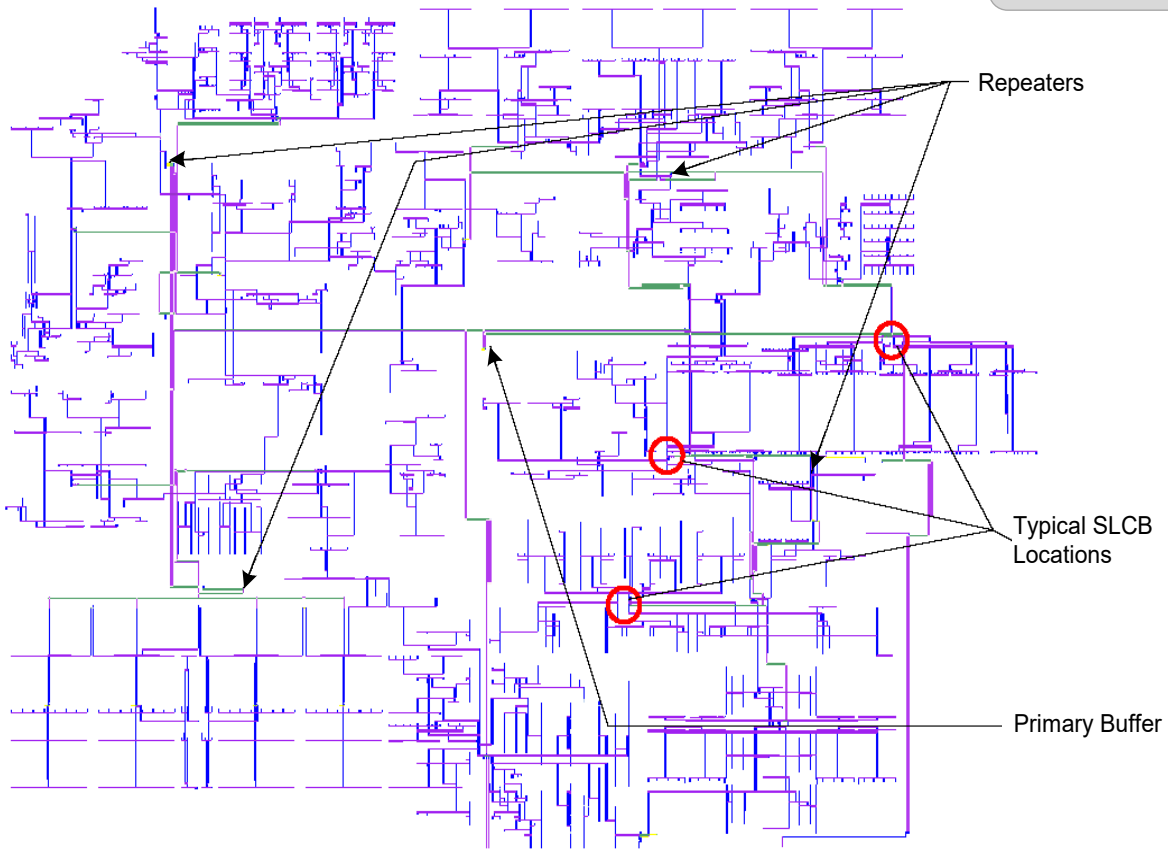
Clock Spine

- IBM PowerPC (2002)



IBM, ISSCC 2000

- Intel Itanium 2 (2005)



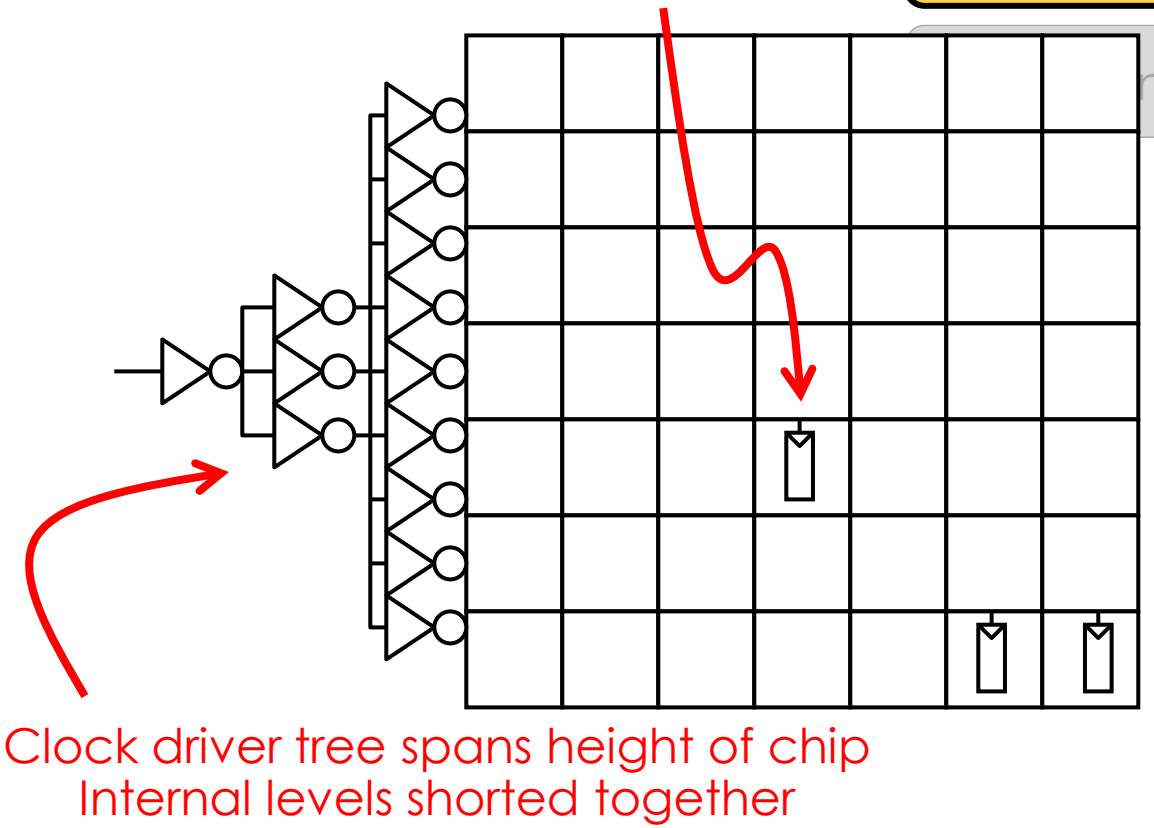
Source: CMOS VLSI Design, 4th Ed.

Lower Skew – Clock Grids

Clock Tree

Clock Grid

- **Advantages:**
  - Skew determined by grid density and not overly sensitive to load position
  - Clock signals are available everywhere
  - Tolerant to process variations
  - Usually yields extremely low skew values
- **Disadvantages**
  - Huge amounts of wiring & power
    - Wire cap large
    - Strong drivers needed – pre-driver cap large
    - Routing area large
  - To minimize all these penalties, make grid pitch coarser
    - Skew gets worse
    - Losing the main advantage
  - Don't overdesign – let the skew be as large as tolerable
  - Still – grids seem non-feasible for SoC's



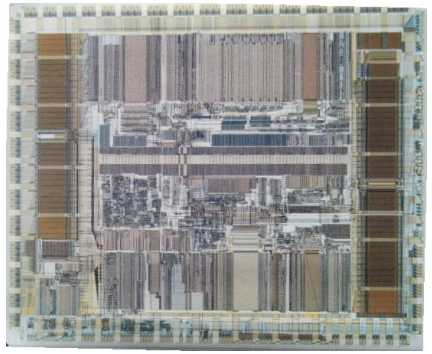
DEC Alpha – Generations of Grids

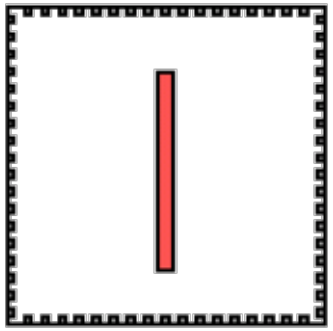
Clock Tree

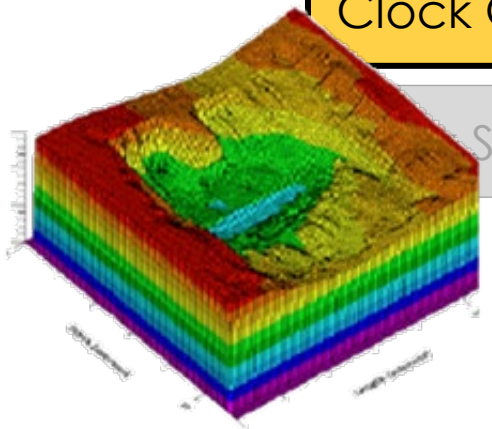
Clock Grid

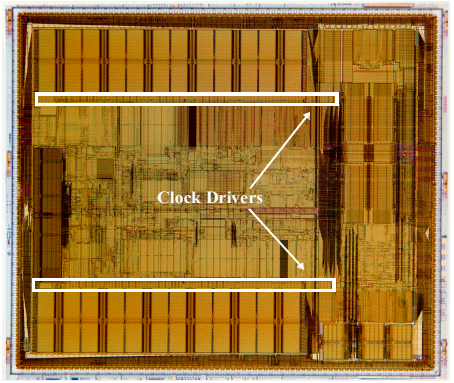
Spine

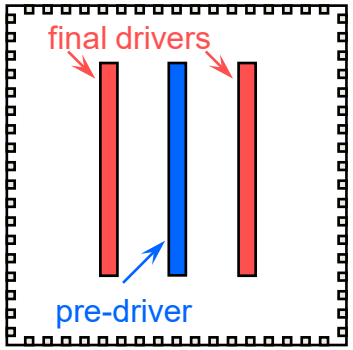
- **21064 (EV4) – 1992**
  - 0.75µm, 200MHz, 1.7M trans.
  - Big central driver, clock grid
    - 240ps skew
- **21164 (EV5) - 1995**
  - 0.5 µm, 300MHz, 9.3M trans.
  - Central driver, two final drivers, clock grid
    - Total driver size – 58 cm!
    - 120ps skew
- **21264 (EV6) - 1998**
  - 0.35 µm, 600MHz, 15.2M trans.
  - 4 skew areas for gating
    - Total driver size: 40 cm
    - 75ps skew

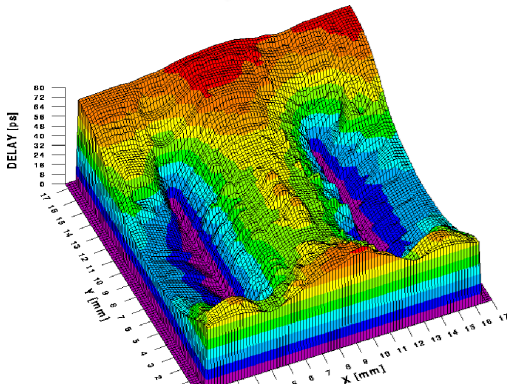


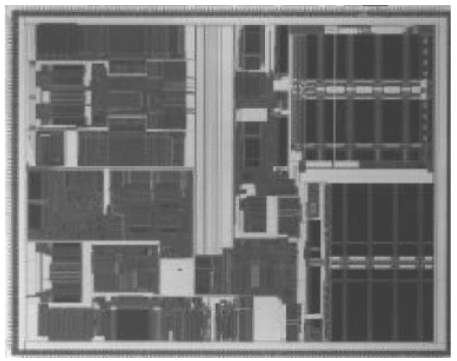


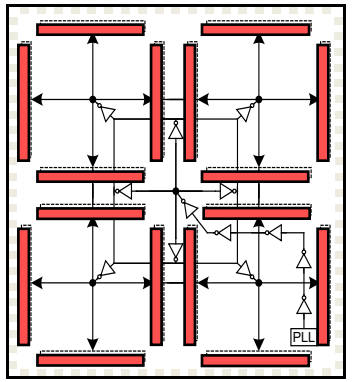


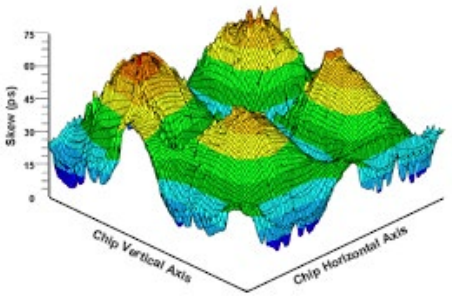












Clock Spines

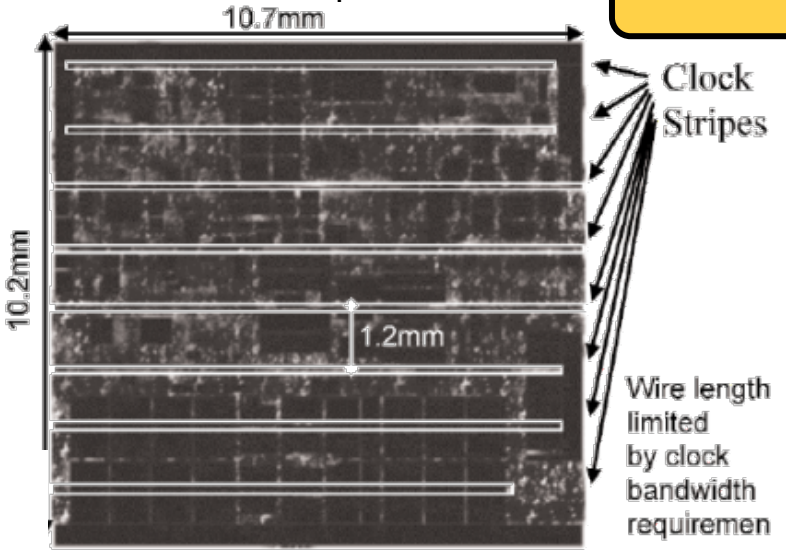
- **Clock grids are too power (and routing) hungry.**
- **A different approach is to use spines**
  - Build an H-Tree to each spine
  - Radiate local clock distribution from spines
- **Pentium 4 (2001) used the clock spine approach.**

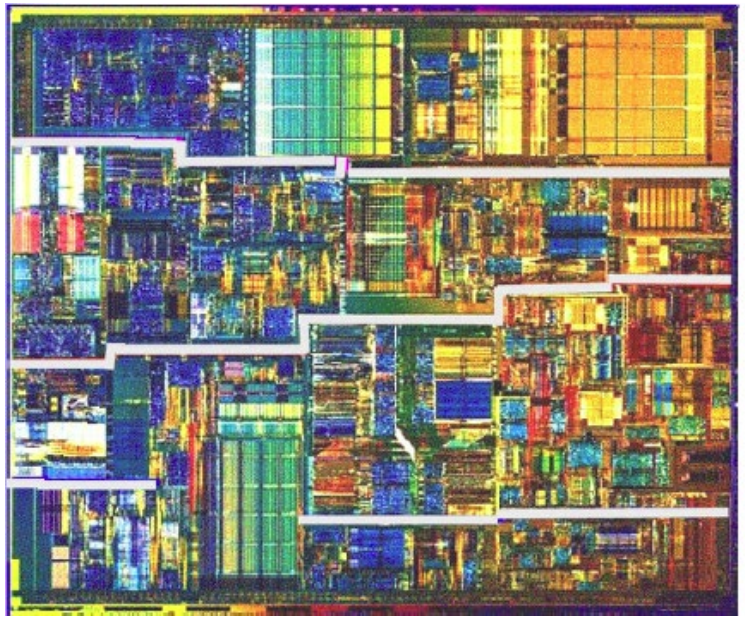
Clock Tree

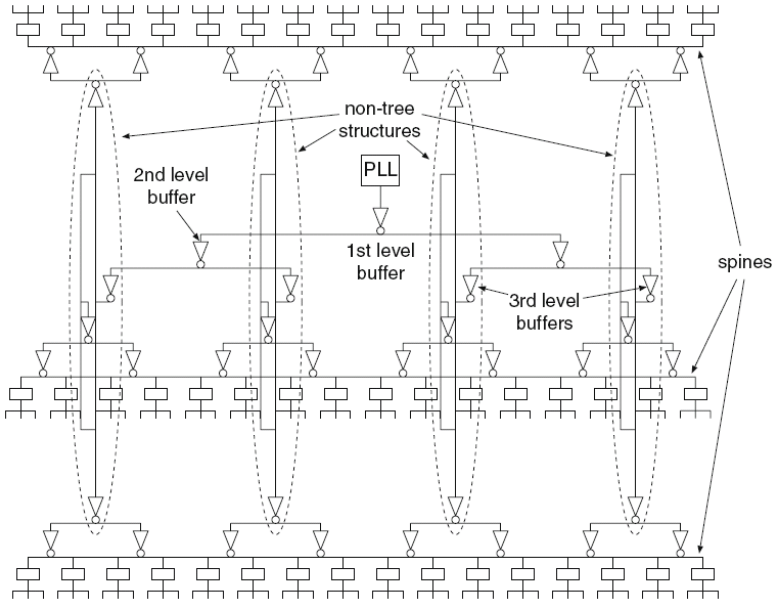
Clock Grid

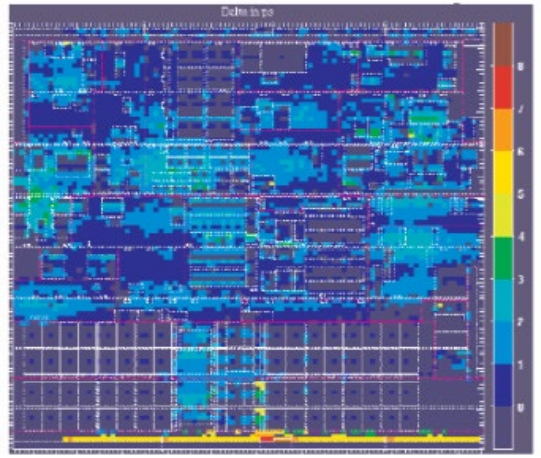
Clock Spine

Later Pentium 4's used more spines









delay in ps

Source: Bindal ISSCC 2003

TEL AVIV UNIVERSITY

Summary of main clock dist. approaches

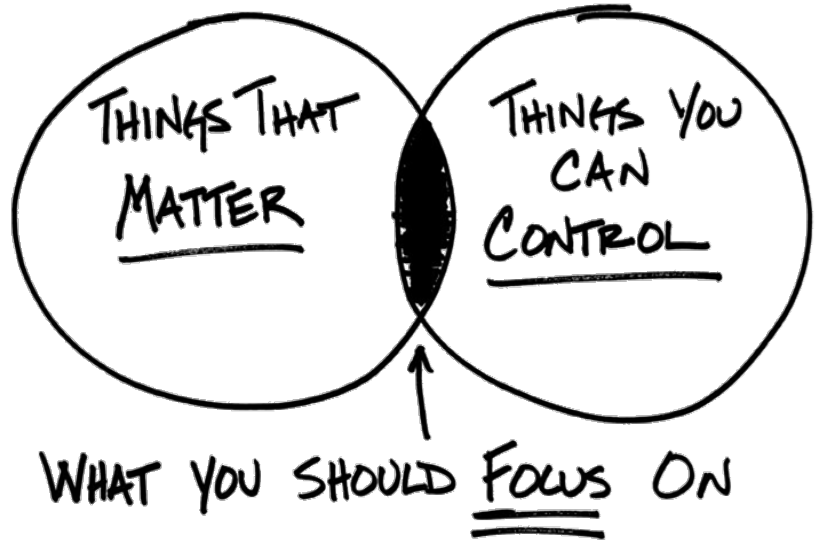
Three basic routing structures for global clock:

- **H-tree**
  - Low skew, smallest routing capacitance, low power
  - Floorplan flexibility is poor.
- **Grid or mesh**
  - Low skew, increases routing capacitance, worse power
  - Alpha uses global clock grid and regional clock grids
- **Spine**
  - Small RC delay because of large spine width
  - Spine has to balance delays; difficult problem
  - Routing cap lower than grid but may be higher than H-tree.

Structure	Skew	Cap/area/ power	Floorplan Flexibility
H-Tree	Low/Med	Low	Low
Grid	Low	High	High
Spine	High	Medium	Medium

Clock Concurrent Optimization

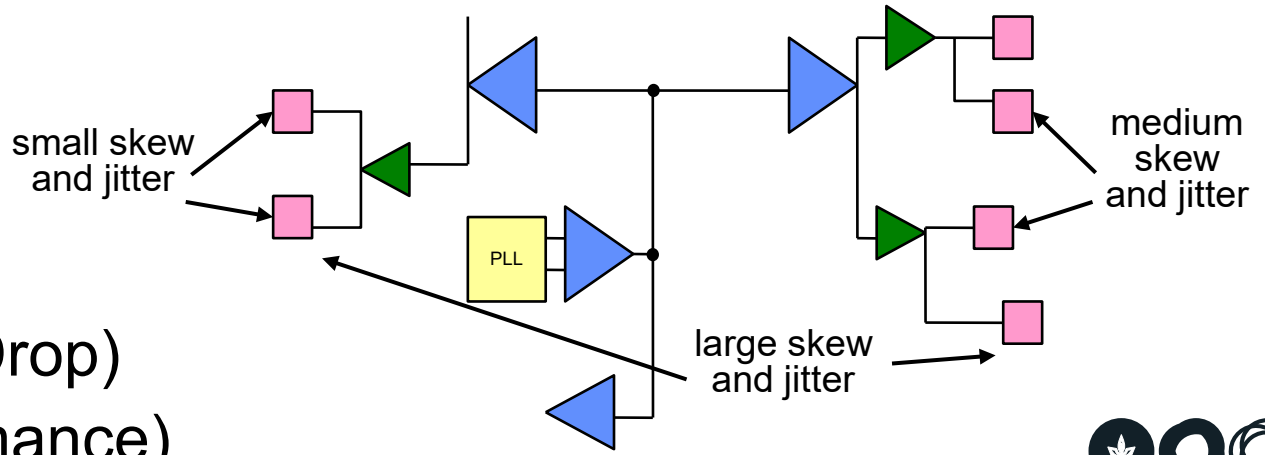
- **What is the main goal of CTS?**
  - We are trying as hard as we can to minimize skew.
  - And on the way, we are burning power, wasting area, suffering from high insertion delay, etc.
  - ***But is minimal skew our actual goal***?
  - Why are we minimizing skew in the first place?

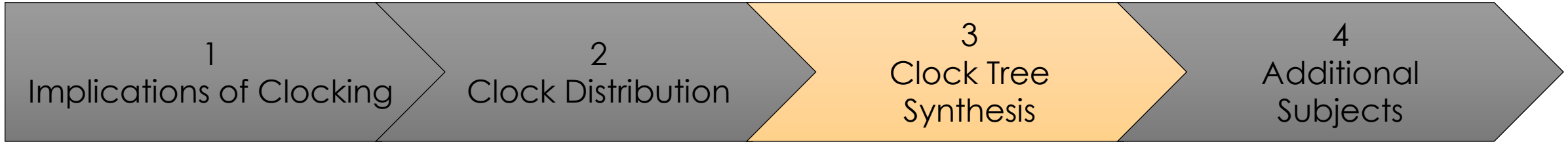


- **Maybe we should forget about skew and focus on our real goals?**
  - We need to **meet timing** and DRV constraints.
  - Minimizing skew was just to correlate post-CTS and pre-CTS timing.
  - But maybe we should just *consider timing*, while building our clock tree.
- **This new approach is known as Clock Concurrent Optimization (CCOpt)**

Clock Concurrent Optimization

- **CCOpt Methodology:**
  - First, build a clock tree, in order to fix DRVs.
  - Then check timing (setup and hold) and fix any violations.
- **Why is this a good approach?**
  - Most timing paths are local.
  - Therefore, they probably come from the same clock branch and don't need much skew balancing to start with.
- **Less skew balancing leads to:**
  - Lower insertion delay (power, jitter)
  - Fewer clock buffers (power, area)
  - Distribution of peak current (less IR Drop)
  - A heavy dose of useful skew (performance)



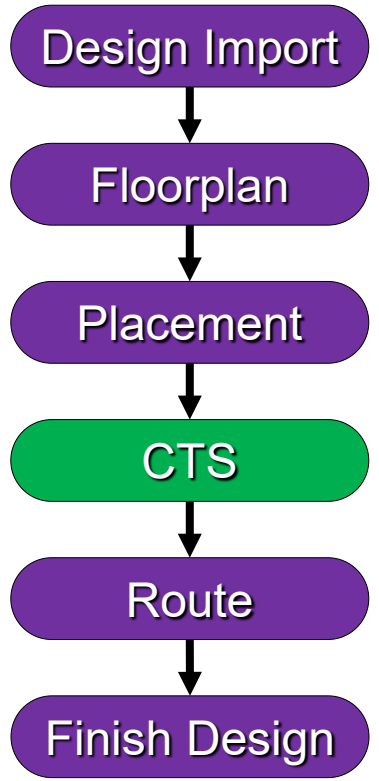


Clock Tree Synthesis in EDA

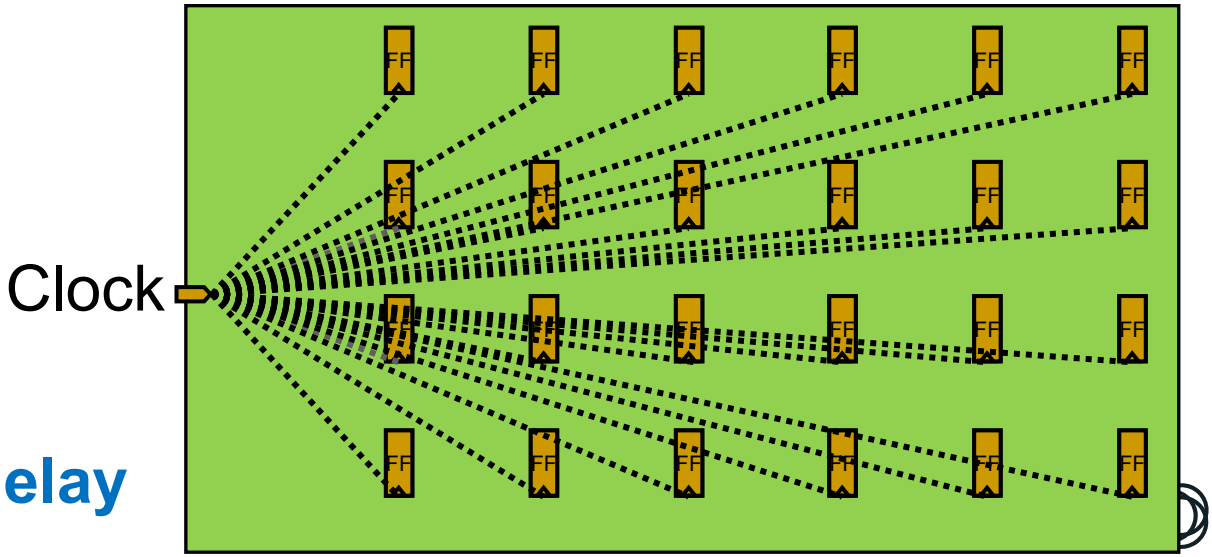
Running CTS

Starting Point Before CTS

DRV = Design Rule Violations



- **So remember:**
  - Our design is placed, and therefore, we know the location of all clock sinks (register and macro clock pins)
  - All clock pins are driven by a single clock source, which we considered ideal until now.
  - We may have several clocks and some logic on the clock network, such as clock gates, muxes, clock dividers, etc.
- **We must now buffer the clock nets to:**
  - Meet **DRV** constraints:
    - Max **fanout**, max **capacitance**, max **transition**, max **length**
  - Meet clocking goals:
    - Minimum **skew**, minimum **insertion delay**

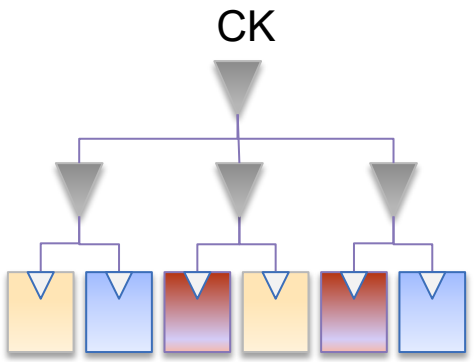


Clock Tree Synthesis Goal and Flows

- **Placement should be completed**
  - Acceptable congestion, setup timing, and logical DRCs
  - High fanout signal nets (reset, scan enable, etc) are buffered
- **The goal of the clock tree synthesis design phase is to:**
  - Build the clock tree buffer structure
  - Route the clock nets
  - Optimize datapath logic for setup and hold timing, power and and DRCs
- **Two CTS flows are supported:**
  - **Classic CTS flow**: CTS first, followed by data path optimization
  - **Concurrent Clock & Data flow (CCD)**: CTS and data path optimization performed concurrently
    - ***CCD is enabled by default*** *during* **`place_opt`** *and* **`clock_opt`**
      CCD is also available during **`route_opt`**

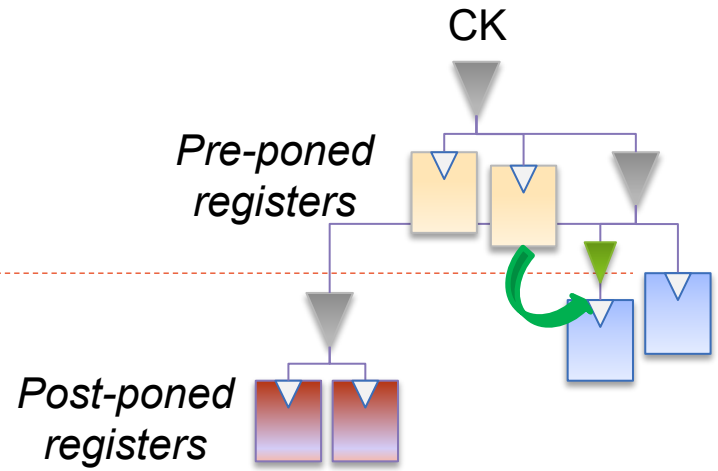
Comparing Classic CTS versus CCD

Classic CTS



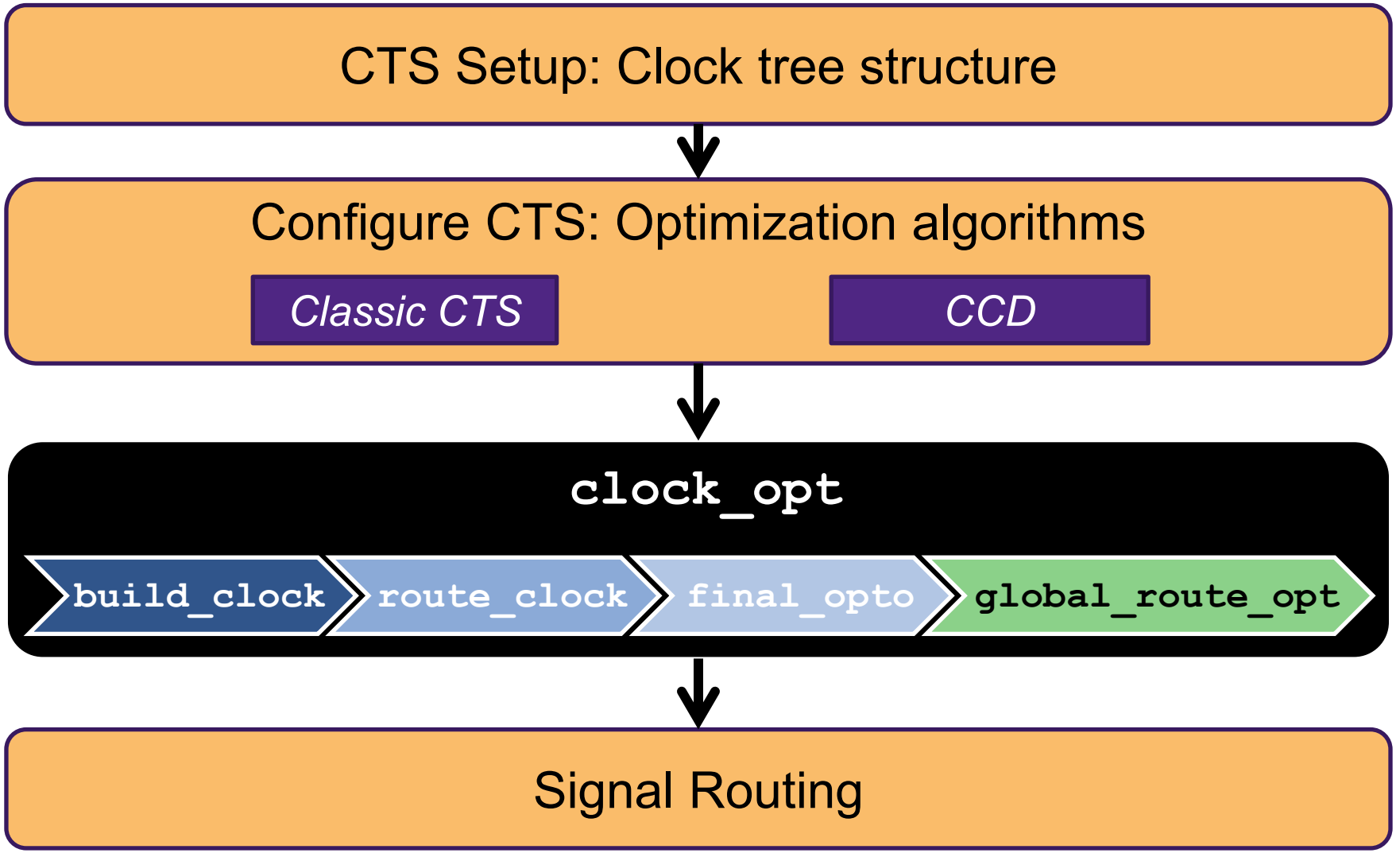
- Clock tree is built while ignoring the data paths
- Goal: Minimize skew
  - Allows full clock-cycle reg-to-reg timing
- Data path is optimized post-CTS
  - Clock tree remains untouched

Concurrent Clock&Data



- Clock tree is built with full knowledge of data path timing
- Goal: Meet setup/hold timing
  - Reg-to-reg timing may be larger or smaller than a clock-cycle
- Data path is optimized along with incremental clock tree modifications *(green buffer)*

Classic CTS and CCD Flow



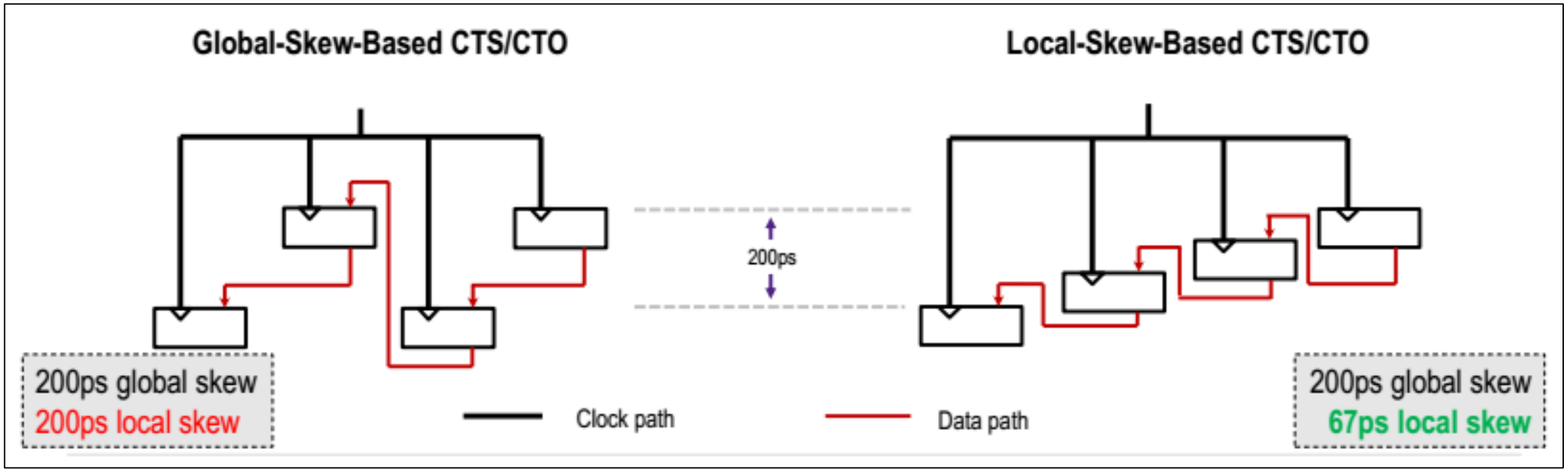
Clock Tree Specific Checks

`check_design` checks for, and suggests solutions to address:

- **Clock Definitions and Propagation**
  - Generated clocks cannot be reached by master clock, etc.
- **Reference Cells**
  - Clock references with `dont_touch`, etc.
- **Skew Balancing**
  - Balancing conflicts between clocks, etc.
- **Multi-Voltage**
  - Clock nets with MV violations, no AON bufs or invs for CTS etc.
- **Capacitance and Transition Constraints**
- **Other Issues:** **`dont_touch`** nets in clock network, etc.

Local Skew Optimization

Classic CTS + CCD Flow



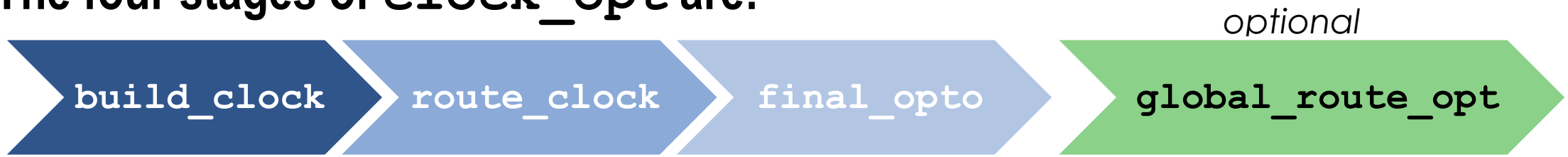
- **Performs timing aware clustering** (as opposed to location based)
- **Optimizes local skew directly for setup and hold timing critical pairs**
- **Can relax skew target where possible** (if slack is positive) **to save clock buffer area**

Classic CTS and CCD Execution

- **Clock tree synthesis, clock tree routing, and data path optimization are all executed by:**

```
clock_opt
```

  - *CCD is enabled by default* (`clock_opt.flow.enable_ccd`)
- **The four stages of `clock_opt` are:**



- **You can control which stages are executed with `-from`/`-to`**
  - For example, to perform only clock tree synthesis (CTS+CTO):

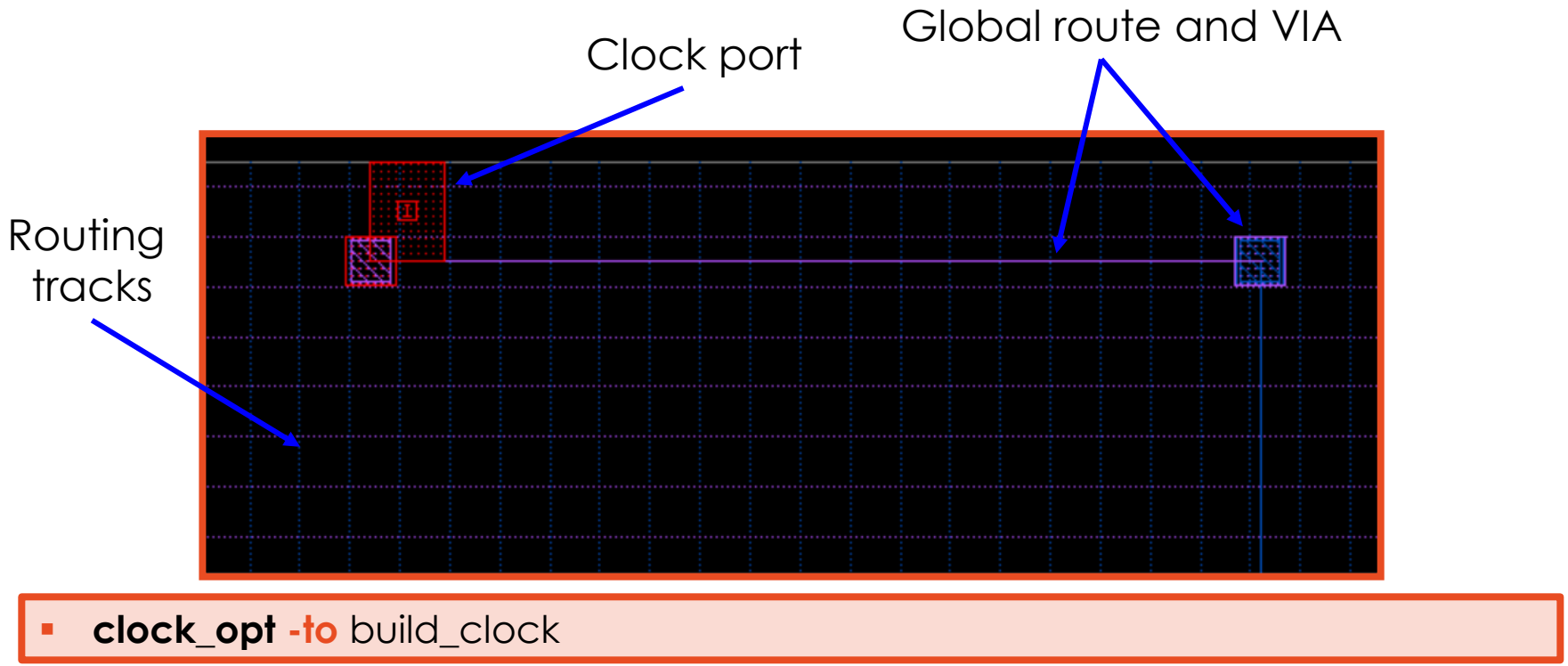
```
clock_opt   -to  build_clock
```

clock_opt Flow

- **clock_opt command is used to synthesize the clock trees, route the clock nets and perform optimization. This command consists of three stages:**
  - build_clock: synthesis of clock trees.
  - route_clock: detail routing of clock nets.
  - final_opto: further design optimization, global routing implementation.

build_clock Stage

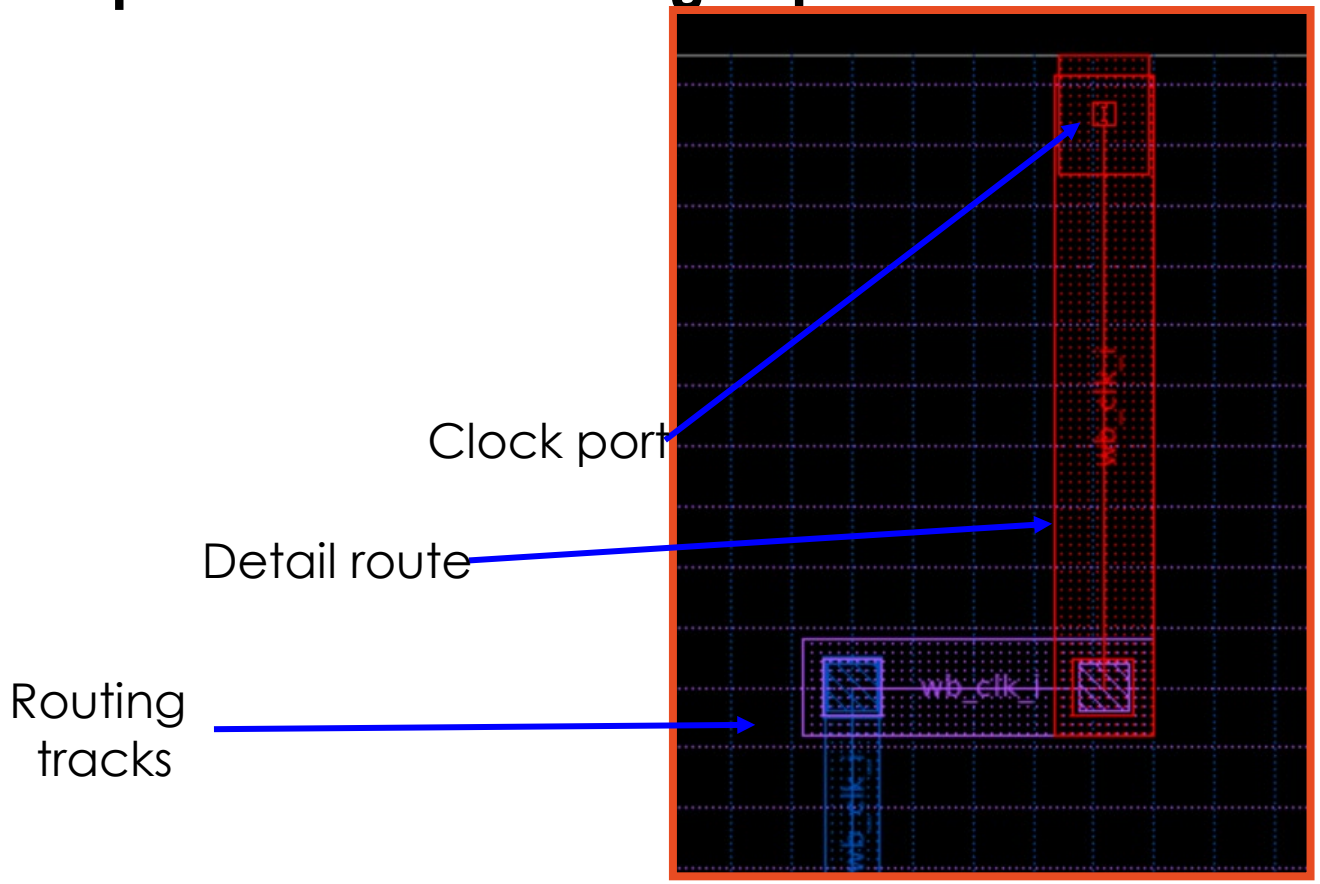
- **Clock trees are synthesized and optimized. Global route for clock nets is performed.**



- **clock_opt -to** build_clock

route_clock Stage

- **Clocks are detail routed and optimized. DRC fixing is performed.**

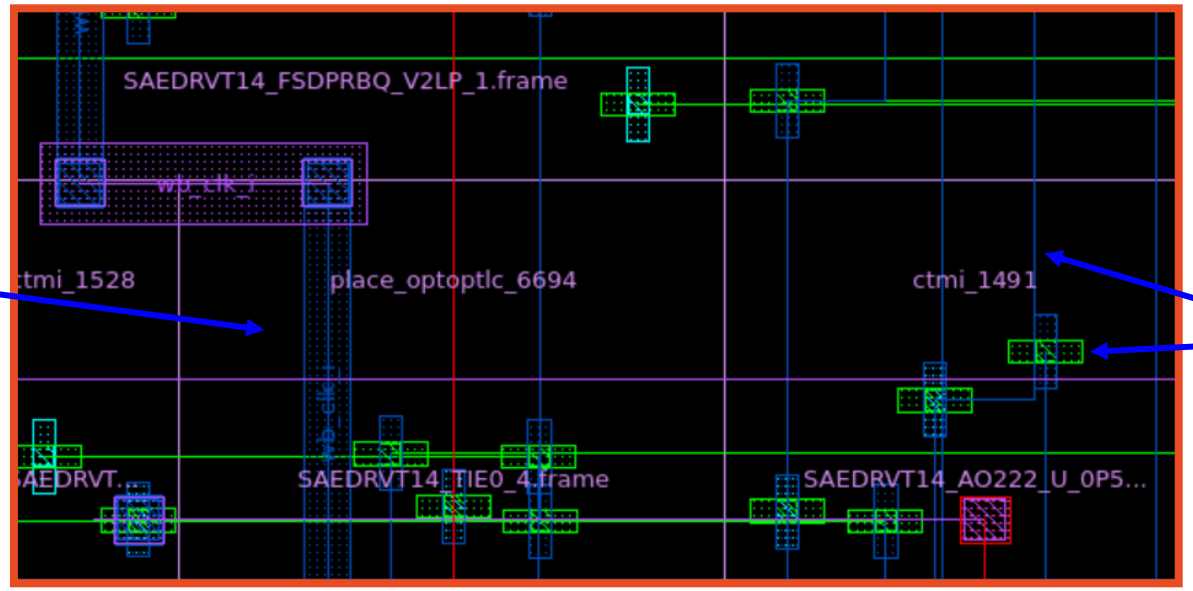


- **clock_opt** -from build_clock -to route_clock

final_opto Stage

- **Further design optimization, timing-driven placement and legalization are performed. The block is global routed for optimization.**

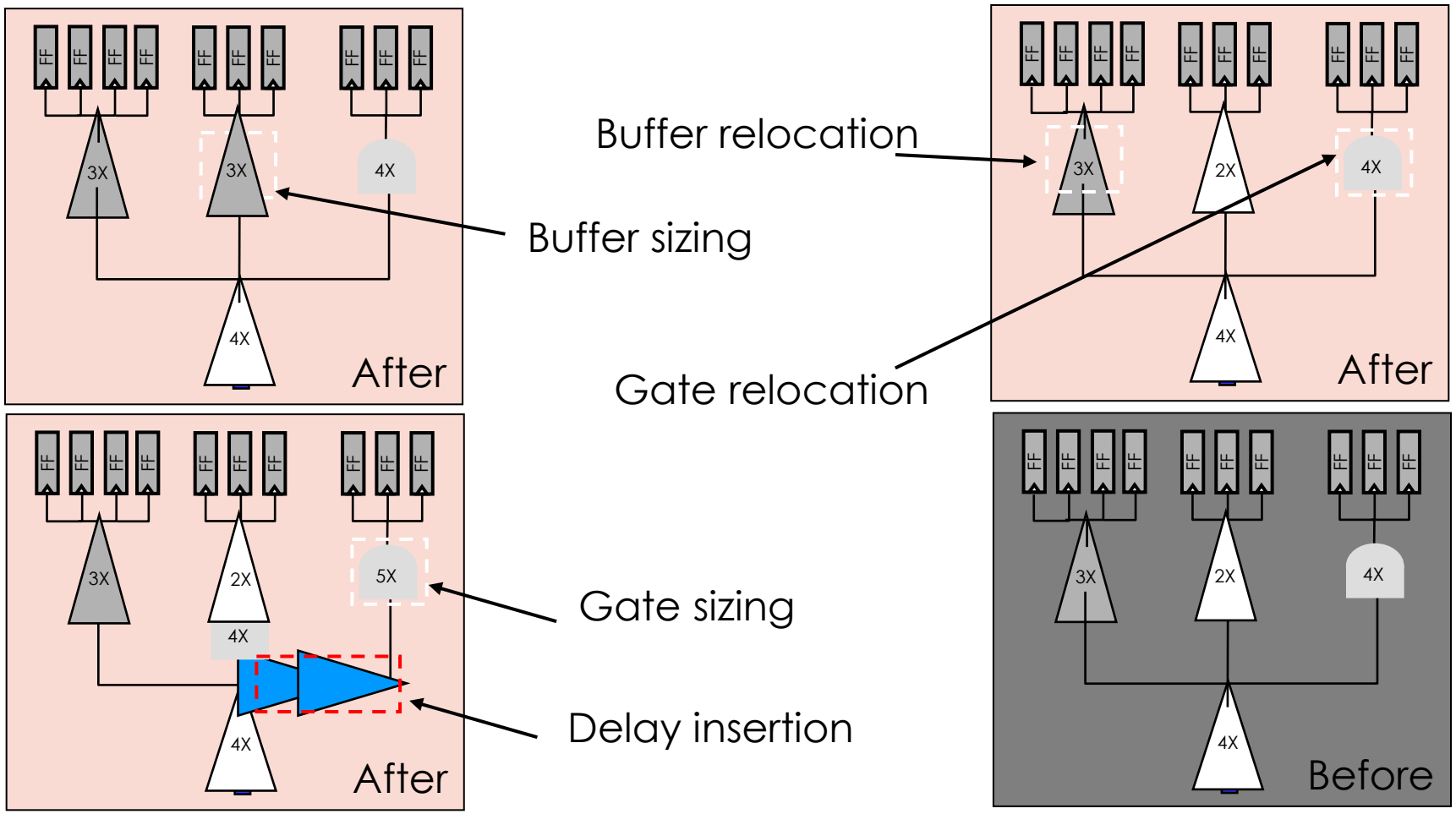
Clock net's Detail route



Signal nets' Global route and VIAs

- **clock_opt -to** final_opto

Clock Tree Optimization



Classic CTS Flow Stages

```
set_app_options -name clock_opt.flow.enable_ccd -value false
```

Default: ***true***

	Command	What does it do?
1	`clock_opt \` `  -to  build_clock`	Builds GR clock trees*, and performs inter-clock balancing, for minimum skew

2	`clock_opt \`
	`  -from route_clock \`
	`  -to  route_clock`
		Routes the clock nets
		
		Updates I/O latencies

3	`clock_opt \` `  -from final_opto`	Performs data path optimization to meet timing and DRCs

Analyzing CTS Results: Clock Timing Report

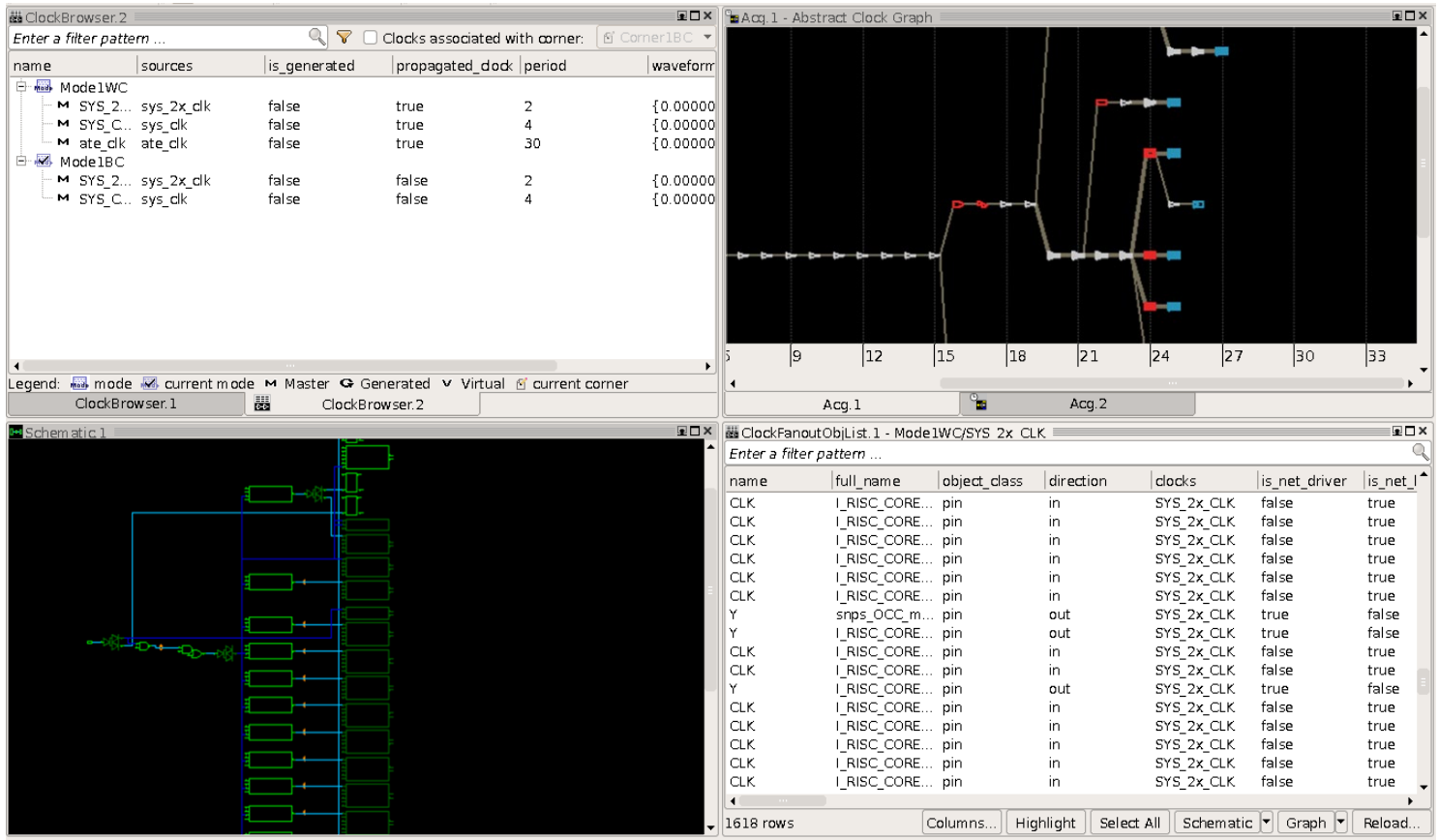
```
report_clock_timing
     -type summary | transition | latency | …
     -modes {m1 m2}
     -corners {c1 c2}   ...
```

- Reports actual, relevant skew, latency, inter-clock latency, etc. for paths that are related
- Includes effects of early/late timing derates
  - `report_clock_qor` does not consider derates
- Example:

```
report_clock_timing -type skew
```

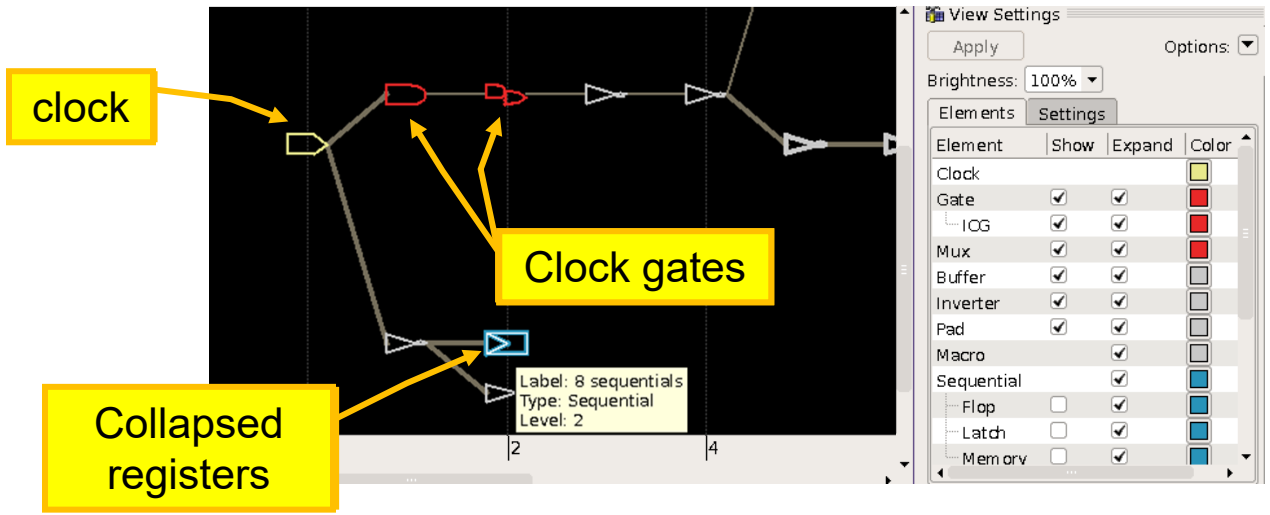
Analysis Using the CTS GUI

- **CTS browser**
  - Identify clock tree object properties and attributes
  - Traverse clock tree levels
- **CTS schematic**
  - Trace clock paths
  - Cross-probe with layout view
- **Clock tree levelized graph**
- **Clock tree latency graph per corner**



GUI Analysis: Clock Tree Levelized Graph

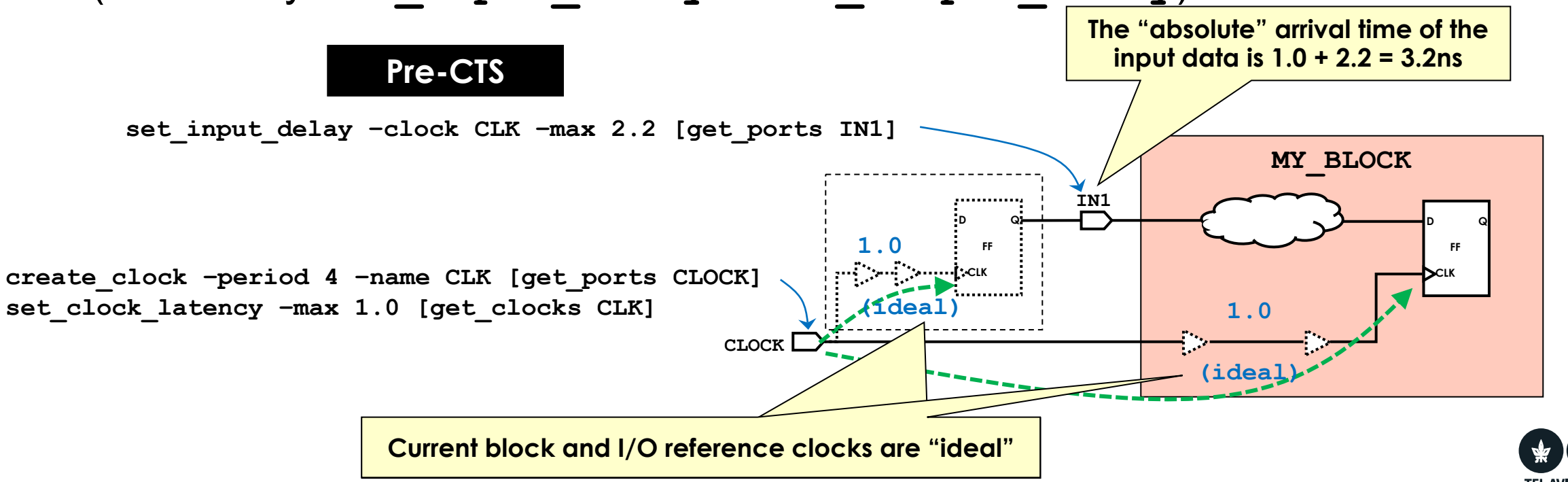
- **Based on the abstraction options selected under the 'Show' column, the graph will show the clock and generated clocks, and collapse the remaining objects. The sinks of the clock tree are collapsed into boxes**



Clock, generated clocks, clock gates, and muxes are shown, the rest is collapsed.

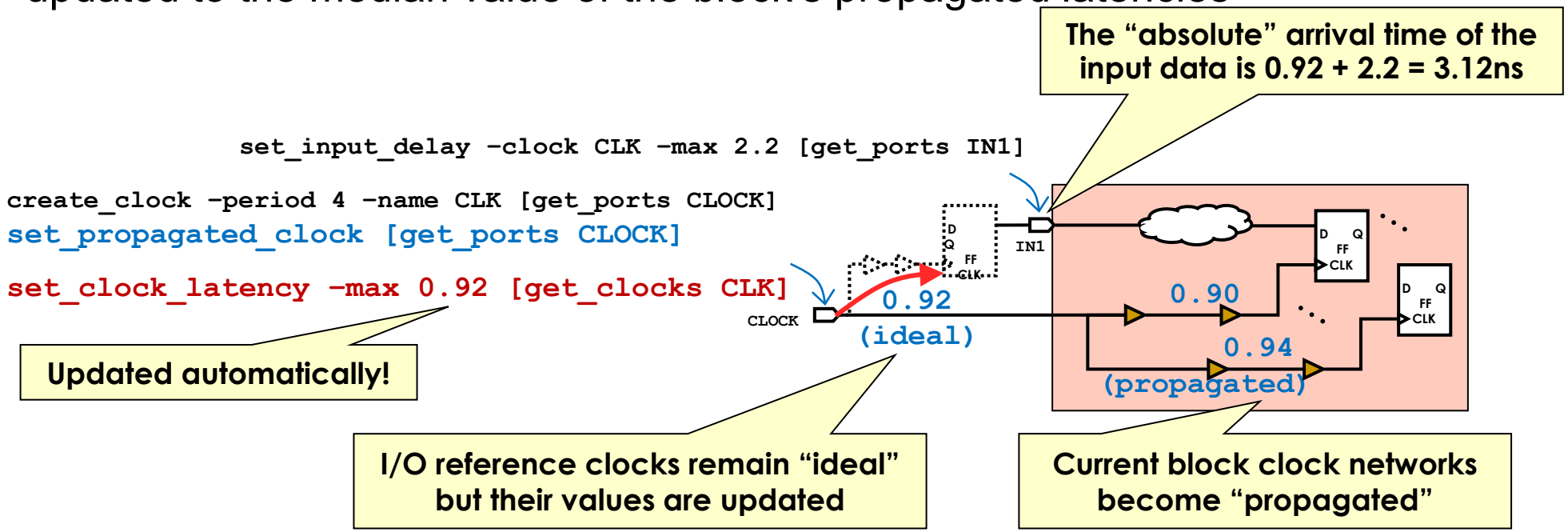
Pre-CTS: Ideal Clock Latencies

- **Prior to CTS, clock networks are "ideal":**
  - Clock latencies are 0ns, by default
  - Timing analysis uses ideal clock latency constraint values (**`set_clock_latency`**)
  - Applies to clock latencies inside the current block and to I/O reference clocks (defined by **`set_input_delay`** / **`set_output_delay`**)



Post-CTS: Propagated Clock Sources / Updated Latencies

- **`clock_opt`'s *build_clock* stage applies `set_propagated_clock` to clock sources (ports/pins), and the *route_clock* stage automatically updates *clock network latencies*:**
  - Actual **propagated** clock network latencies calculated inside the current block
  - Updated **ideal** clock network latencies apply to I/O reference clocks
    - updated to the median value of the block's propagated latencies



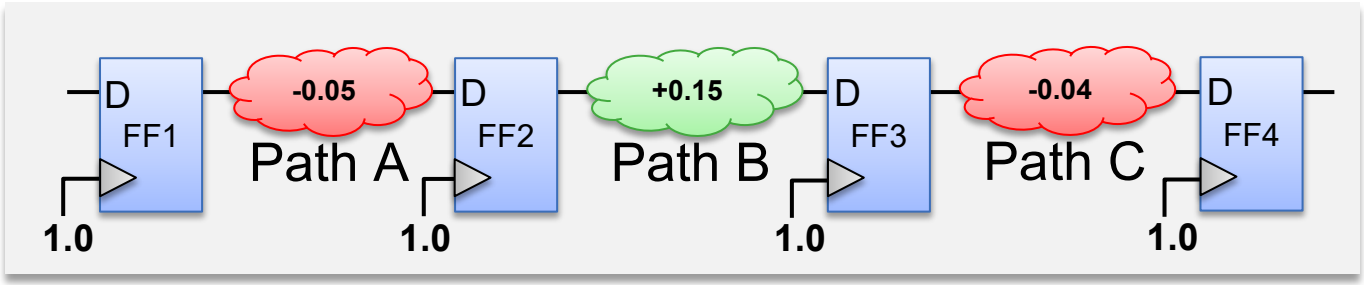
CCD during `place_opt` Uses Ideal Latencies for Skewing

- **Applies *clock network latencies* to <u>all clock pins</u>, individually adjusted to improve timing**

Timing paths just before CCD optimization occurs

User-defined latency constraint on clock:

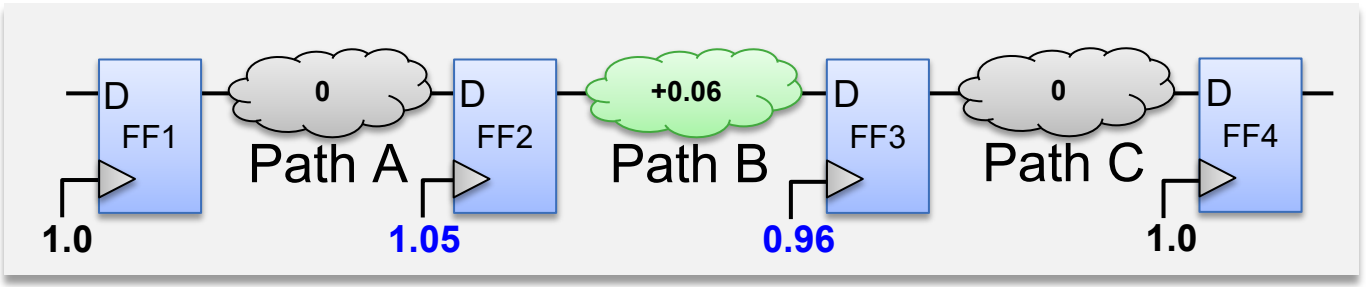
```
set_clock_latency 1.0 [get_clocks CLK]
```



CCD-created latency constraints use the **-offset** switch (pin constraint overrides clock constraint):

```
set_clock_latency  0.05 -offset [get_pins FF2/clk]
set_clock_latency -0.04 -offset [get_pins FF3/clk]
```

CCD increases FF2 latency and decreases FF3 latency (useful skew) to reduce or eliminate violations

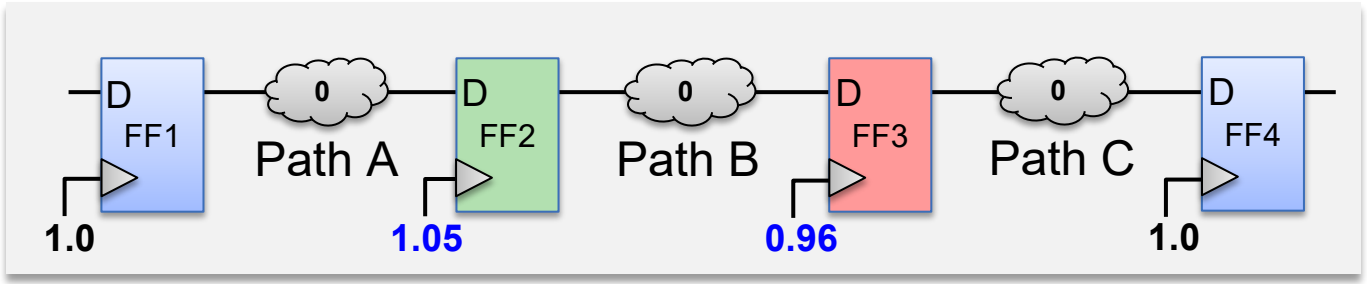


Clock Balance Point Delays

- **Clock pin *network latency constraints* applied by CCD affect pre-CTS data path timing analysis and optimization, but they are ignored by CTS**
- **CCD optimization, therefore, also applies *clock balance point delays* which are used by CTS to implement the intended useful skews**
  - Clock balance point delays are normalized to zero:

Pins that are postponed with respect to 0 (arrive after 0) receive a negative delay

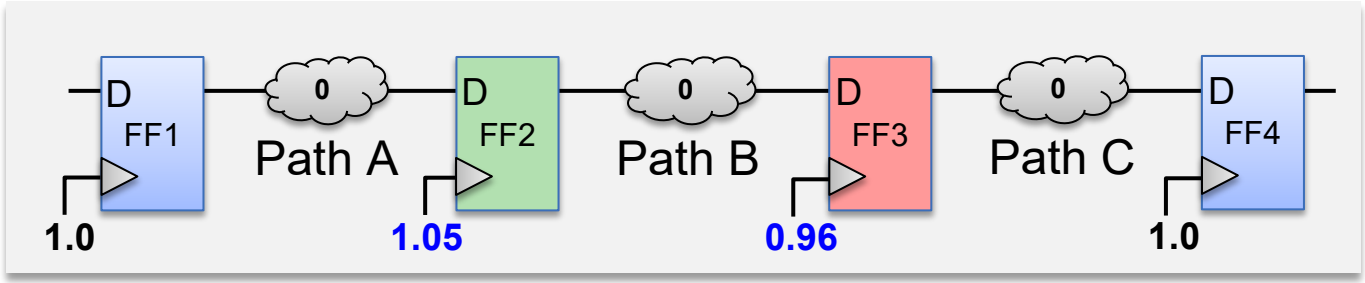
Pins that are "preponed" with respect to 0 (arrive before 0) receive a positive delay




```
set_clock_latency  0.05 -offset [get_pins FF2/clk]
set_clock_latency -0.04 -offset [get_pins FF3/clk]
```

```
set_clock_balance_points -rise -delay -0.05 \
  -offset -balance_points [get_pins FF2/clk]

set_clock_balance_points -rise -delay  0.04 \
  -offset -balance_points [get_pins FF3/clk]
```

Reporting Clock Pin Latencies and Balance Point Delays




```
set_clock_latency  0.05 -offset [get_pins FF2/clk]
set_clock_latency -0.04 -offset [get_pins FF3/clk]
```

```
set_clock_balance_points -rise -delay -0.05 \
  -offset -balance_points [get_pins FF2/clk]

set_clock_balance_points -rise -delay  0.04 \
  -offset -balance_points [get_pins FF3/clk]
```

report_timing -from FF2/CLK -to FF3/D

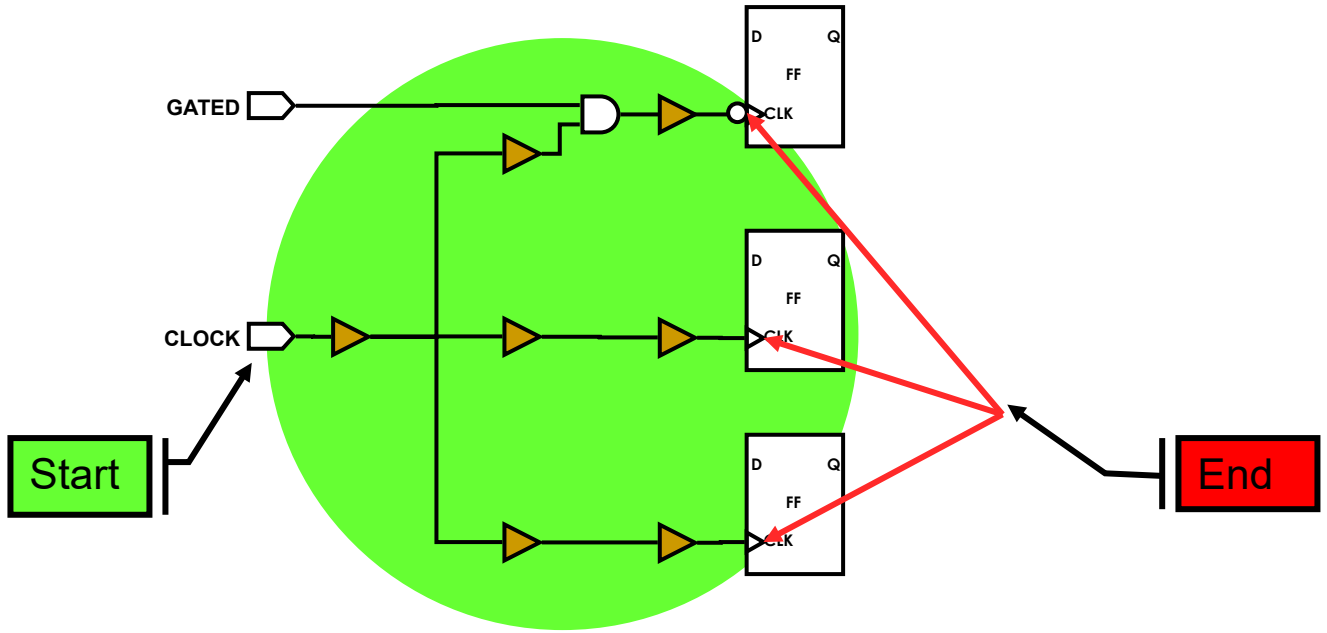
```
  Point                                   Incr       Path
  ------------------------------------------------------
  clock CLK (rise edge)                   0.00       0.00
  clock network delay (ideal)             1.05       1.05

  FF2/clk (SDFFARX1_SVT)                  0.00       1.05 r
  ...
  data arrival time                                  2.61

  clock CLK (rise edge)                   2.00       2.00
  clock network delay (ideal)             0.96       2.96
  FF3/clk (SDFFARX1_SVT)                  0.00       2.96 r
  clock uncertainty                      -0.15       2.81
  library setup time                     -0.20       2.61
  data required time                                 2.61
```

report_clock_balance_points

```
Clock 'CLK':
Balance points:
                ... -------------- OFFSET ---------------
                ... Early-   Early-   Late-    Late-
Point           ...  Rise     Fall     Rise     Fall    Corner
-------------------------------------------------------------
FF2/clk         ... -0.05      --     -0.05      --     ss_125c
FF3/clk         ...  0.04      --      0.04      --     ss_125c
...
```

Setting Up CTS

Where Does the Clock Tree Begin and End?



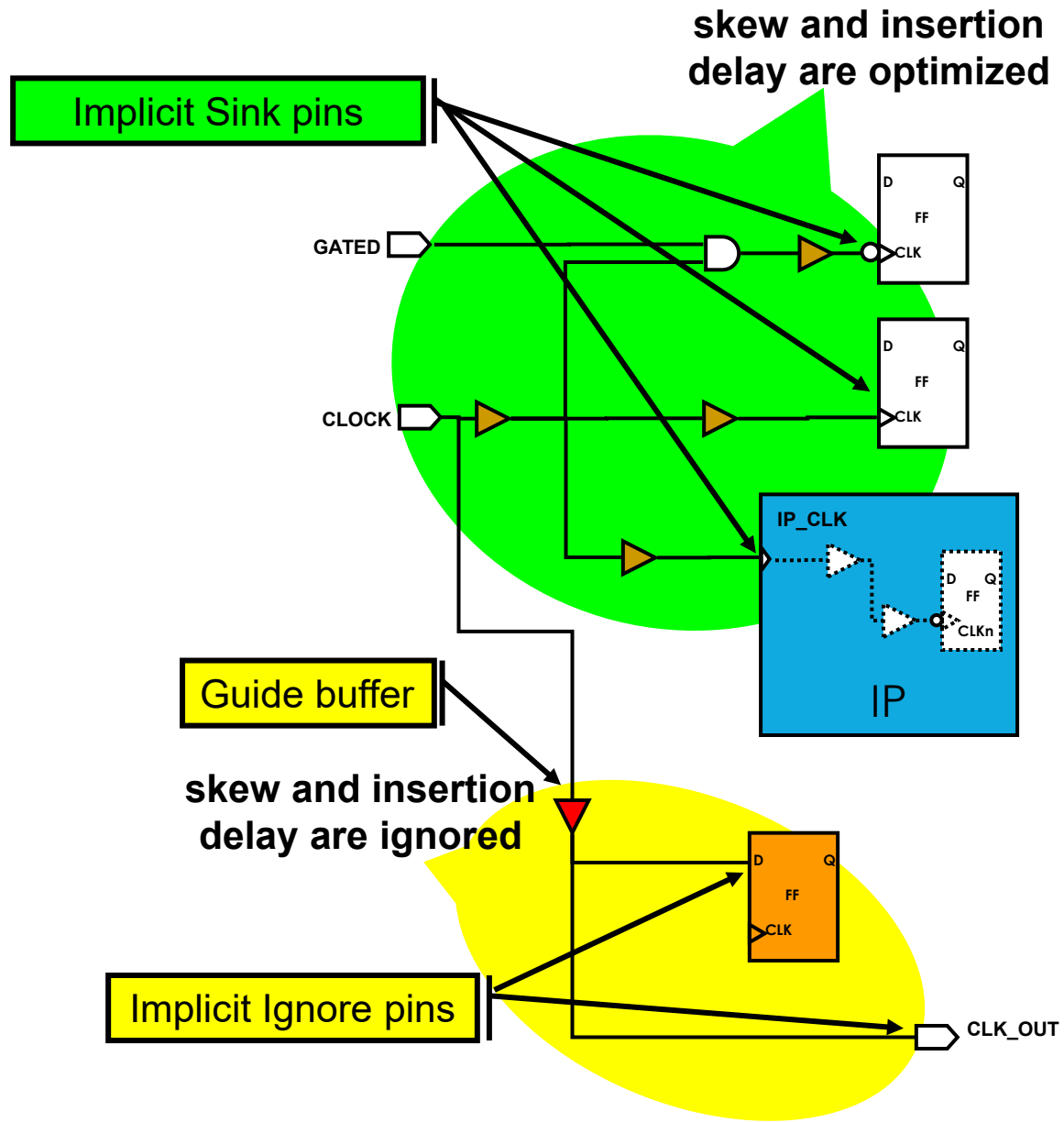
Clock trees start at their "source" defined by

`create_clock` …

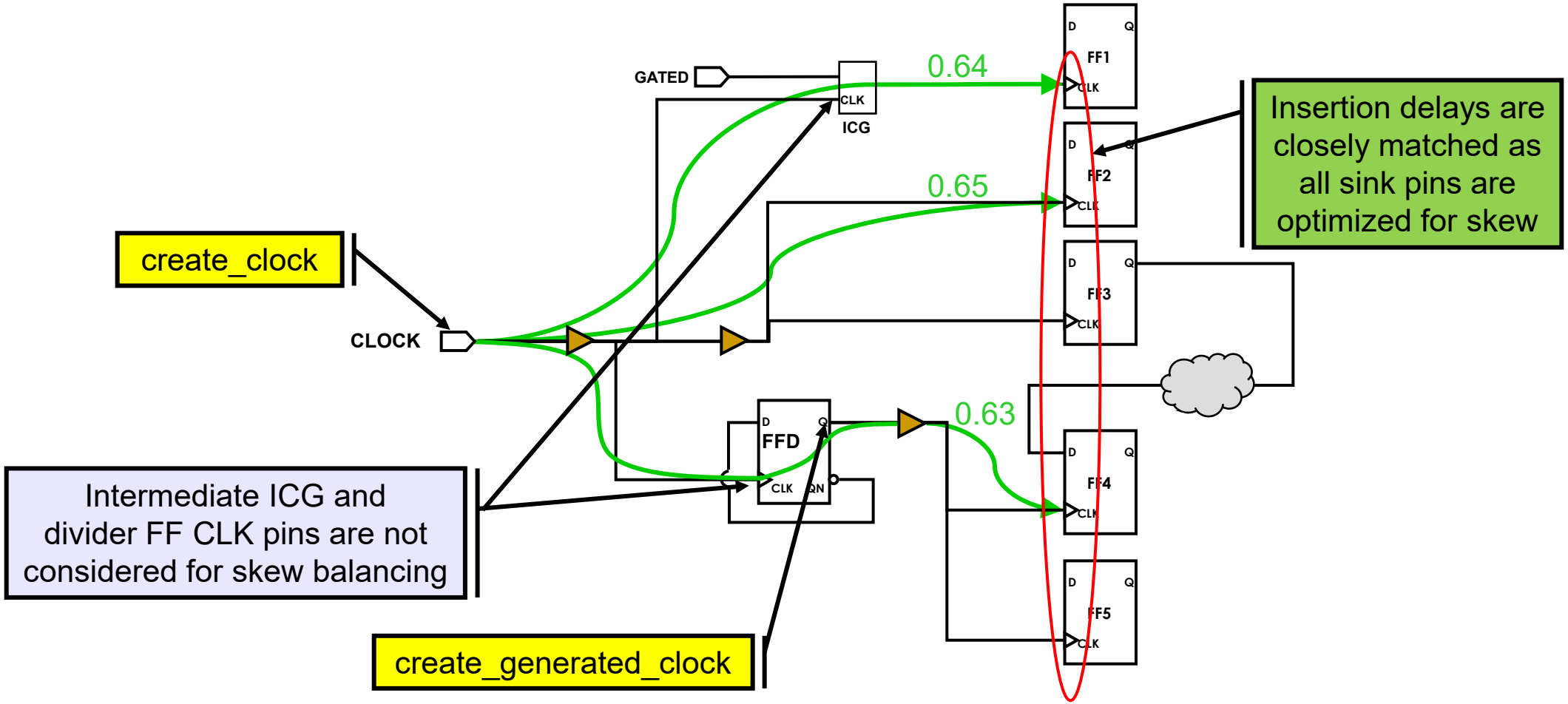
Synthesizable clock trees end (by default) on clock pins of registers or macros

Balance Points: *Sink* and *Ignore* Pins

- **Sink Pins:**
  - CTS optimizes for DRC and clock tree targets (skew, insertion delay)
  - Considers internal insertion delays
- **Ignore Pins:**
  - CTS ignores skew and insertion delay targets
  - CTS inserts a guide buffer
    - Only DRCs fixed after guide buffer



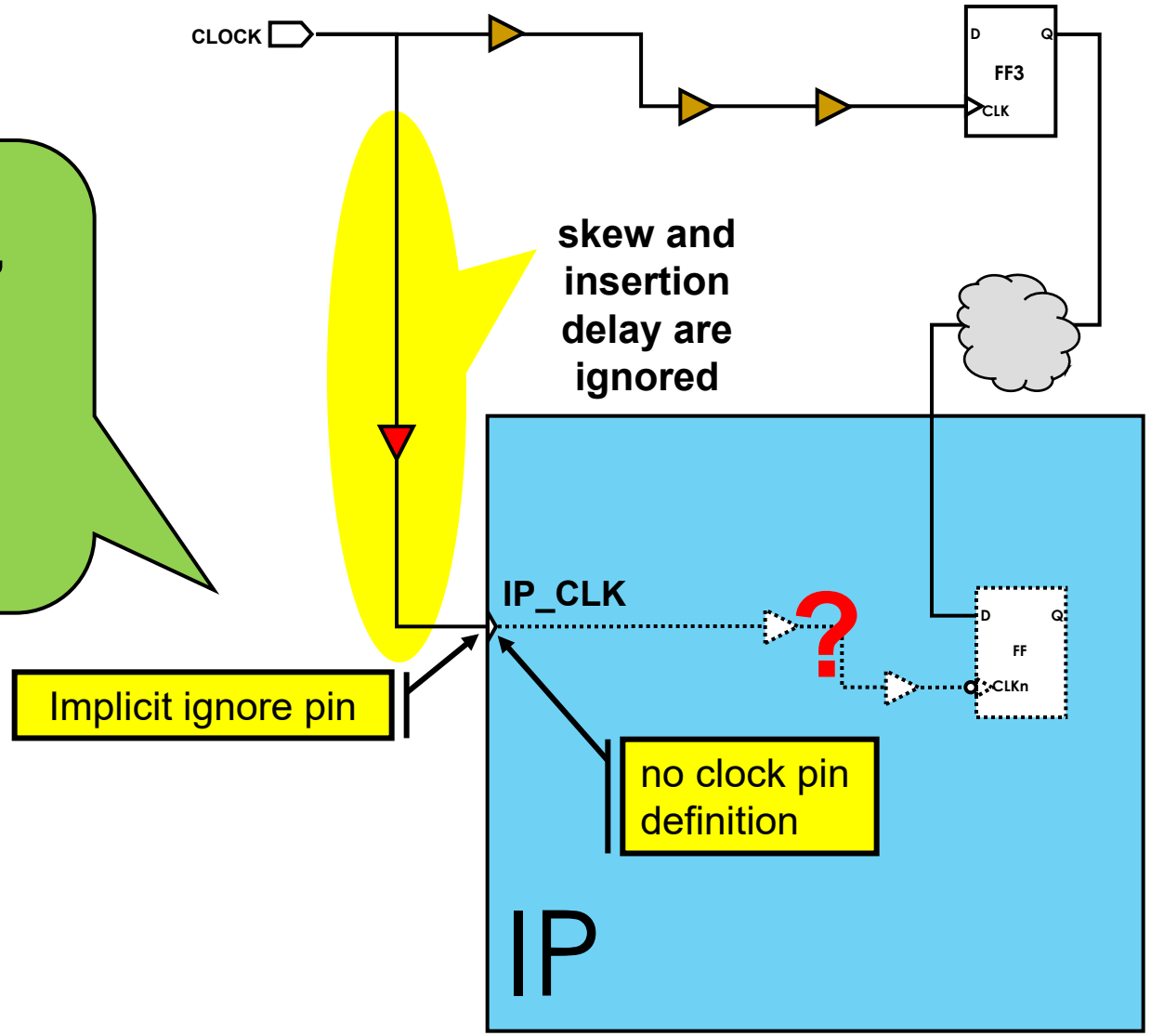
Generated and Gated Clocks



Master and generated clocks are part of the same clock domain.
Skew is balanced globally (across all clock pins) within each clock domain.

User-defined or Explicit Sink Pins

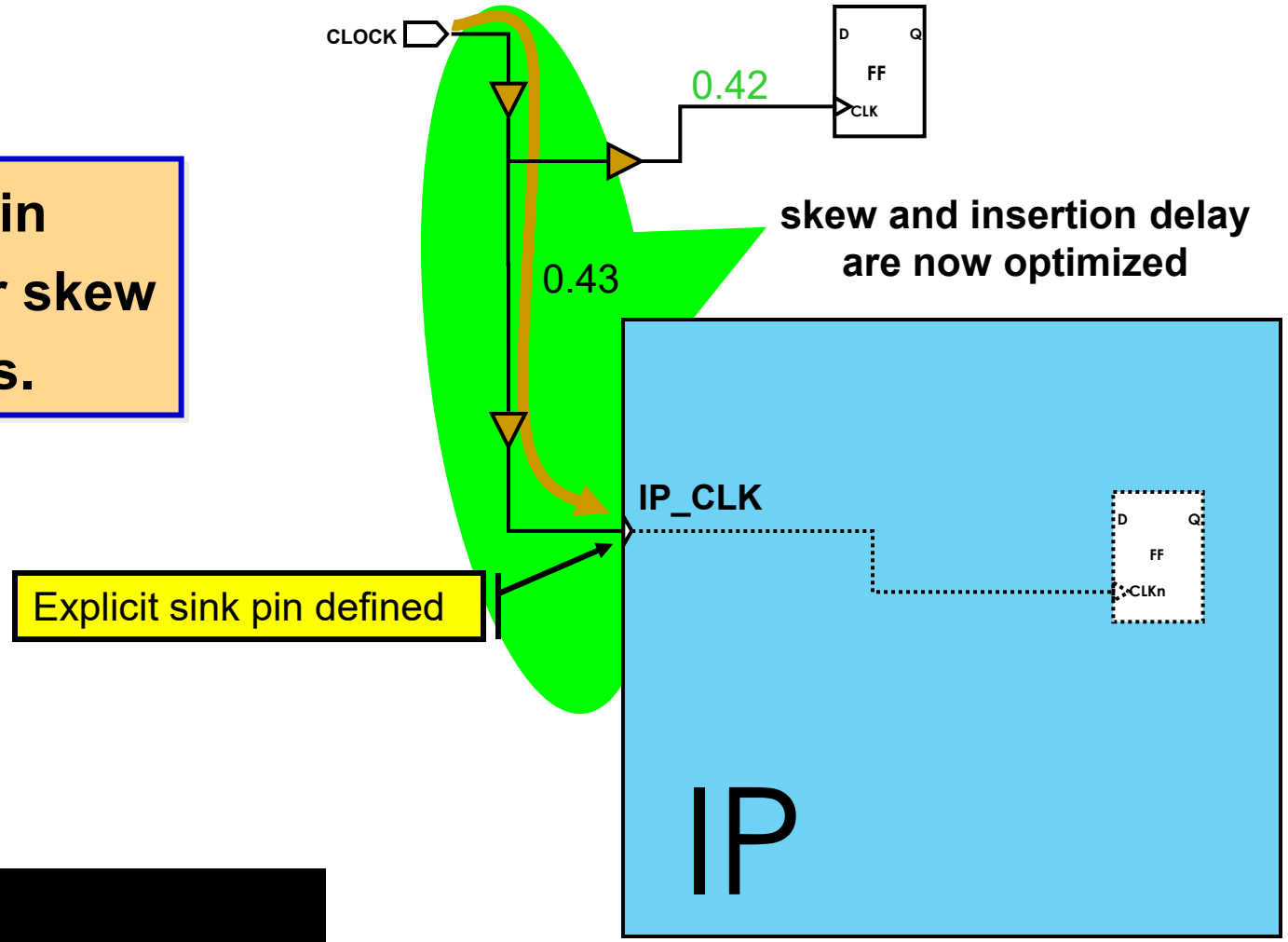
Scenario: If a macro cell clock pin is correctly defined, CTS will treat that pin as an implicit sink pin. In this example the clock pin is not correctly defined.
What will CTS do?



The macro's clock pin is marked as an implicit ignore pin – no skew optimization!

Defining an Explicit Sink Pin

Defining an explicit sink pin allows CTS to optimize for skew and insertion delay targets.

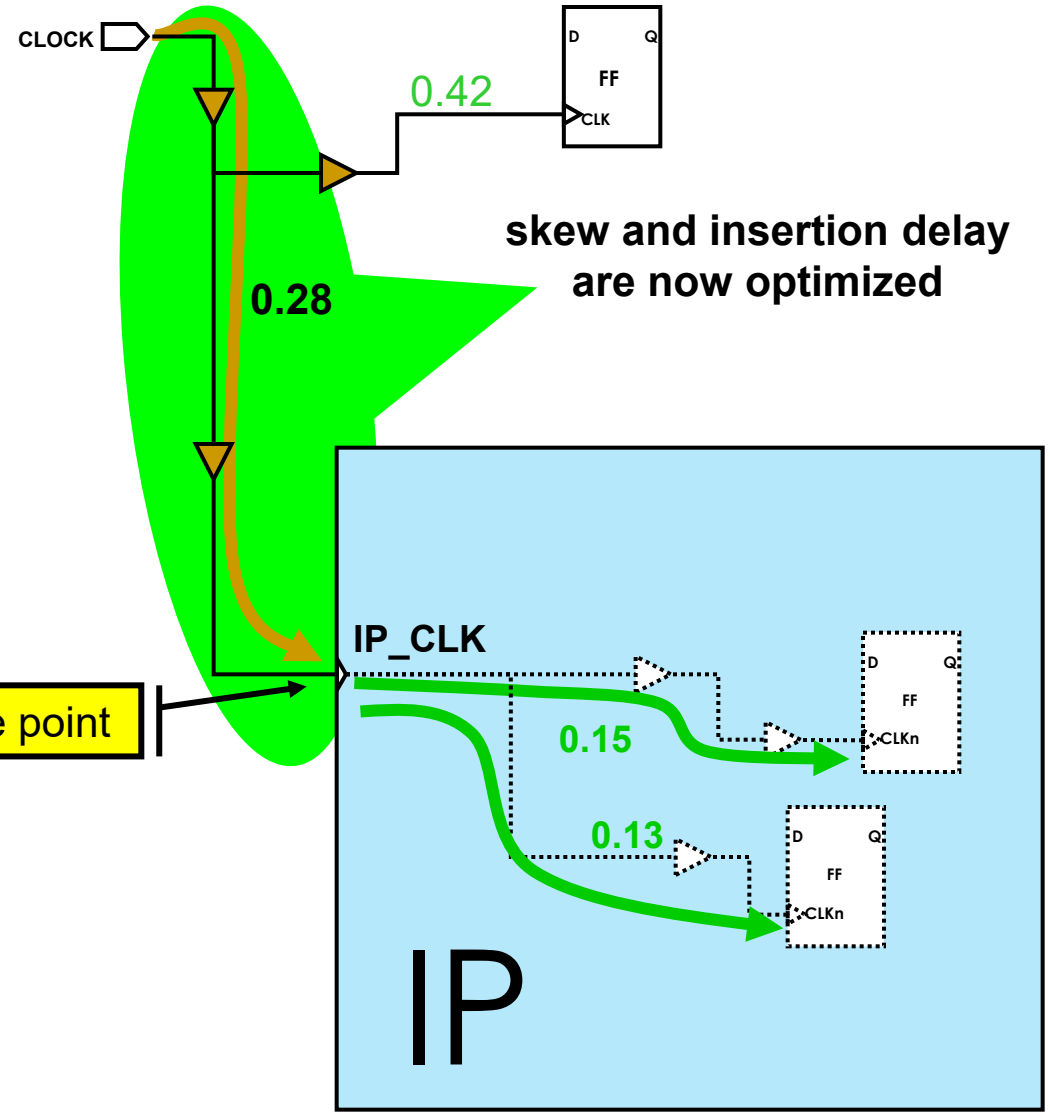


CTS has no knowledge of the IP-internal clock delay – it can only "see" up to the sink pin!

```
set_clock_balance_points      \
       -balance_points [get_pins IP/IP_CLK]
```

Defining an Explicit Sink Pin With Delay

Defining an explicit sink pin with <u>delay</u> allows CTS to adjust the insertion delays based on the specification.



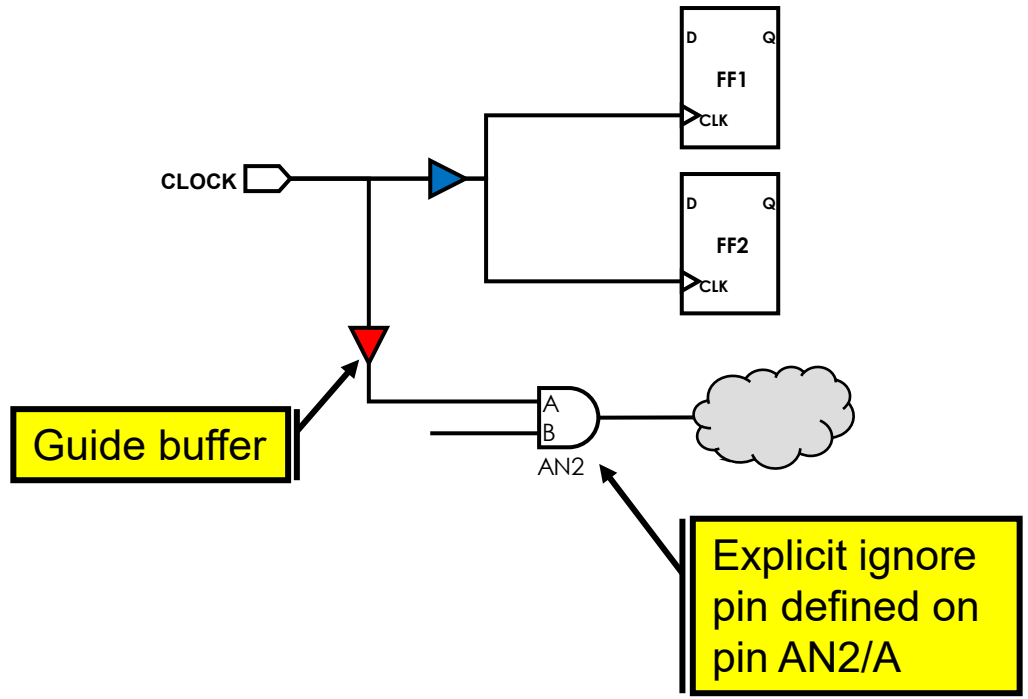

```
set_clock_balance_points          \
  -corner ss125c                  \
  -delay 0.15 -late               \
  -balance_points IP/IP_CLK
```

Specifying a *positive* delay *pre*pones,
specifying a *negative* delay *post*pones the clock pin

Defining an Explicit Ignore (or Exclude) Pin

Explicit ignore pins cause CTS to ignore this pin and down-stream sinks for skew balancing.

```
set_clock_balance_points \
  -clock CLOCK \
  -consider_for_balancing false \
  -balance_points [get_pins AN2/A]
```

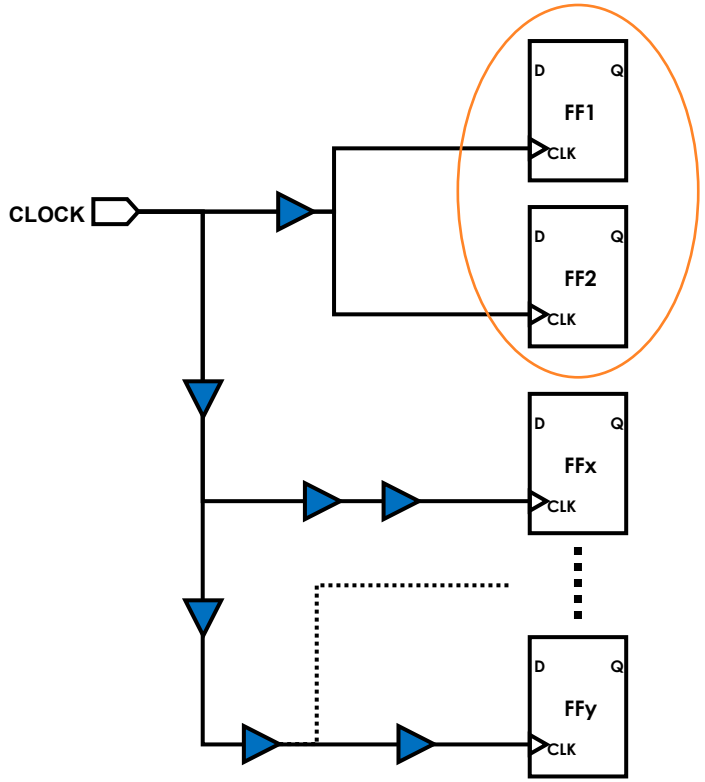


→ CTS inserts a guide buffer; CTS engine fixes DRCs only along the clock path past ignore pins.

Defining a Clock Skew Group

Define a clock skew group if you want to balance a group of clock pins independently from the rest of the clock tree.

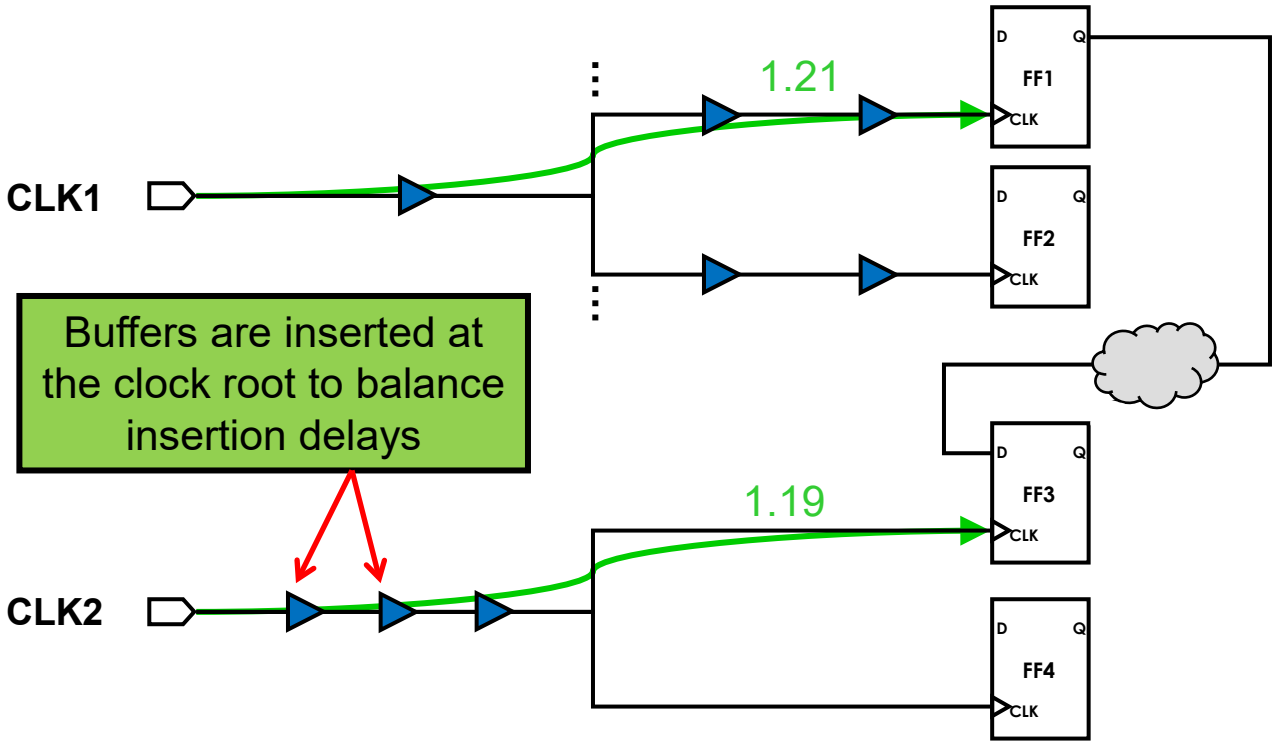
The target skew will be the same as the master clock of this skew group. You cannot set the skew independently.




```
create_clock_skew_group -mode TEST -objects {FF1/CLK FF2/CLK}
```

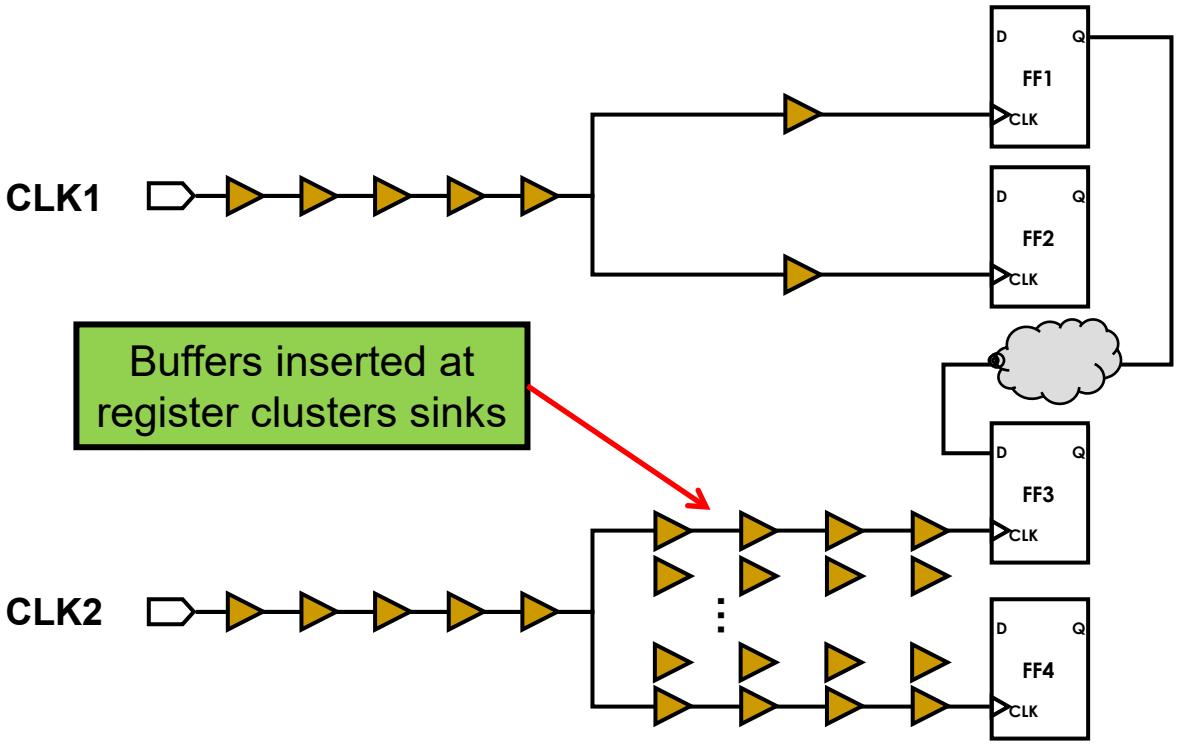
Balancing Occurs During `clock_opt`

Balancing occurs during the `build_clock` stage



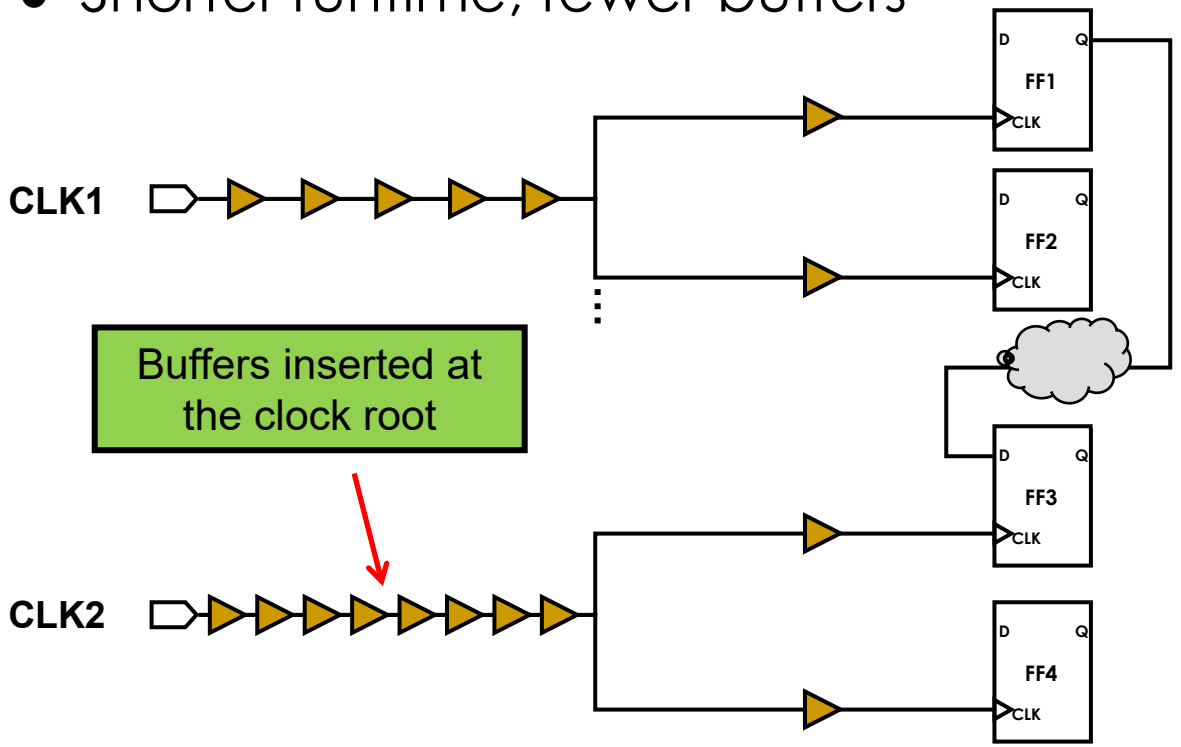
Inter-Clock Balancing is Important for CCD

- **Without inter-clock balancing, CCD will try to meet timing by skewing individual sinks**
  - Longer runtime, many more buffers



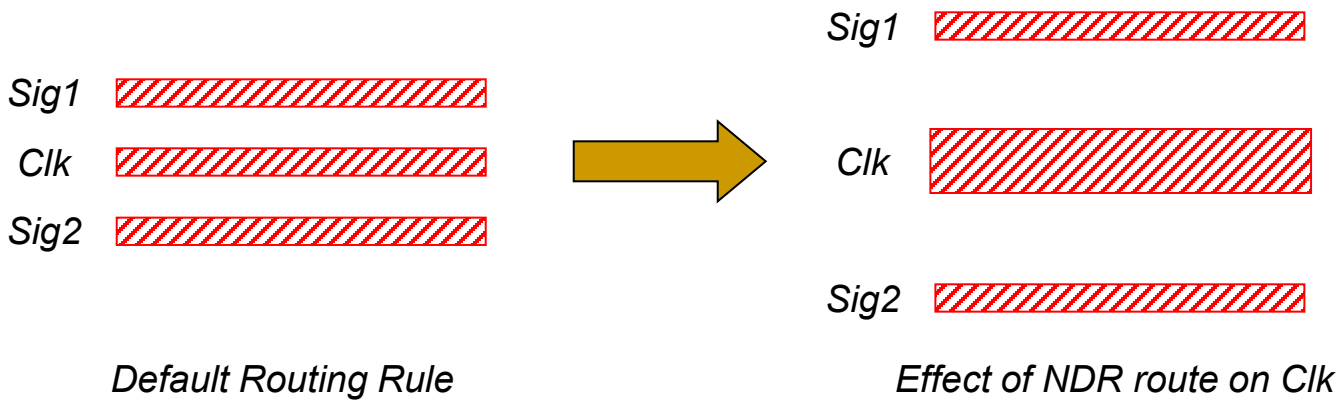
Exaggerated example!

- **With inter-clock balancing, CTS inserts buffers at the root**
  - CCD fine-tunes individual sinks afterwards
  - Shorter runtime, fewer buffers



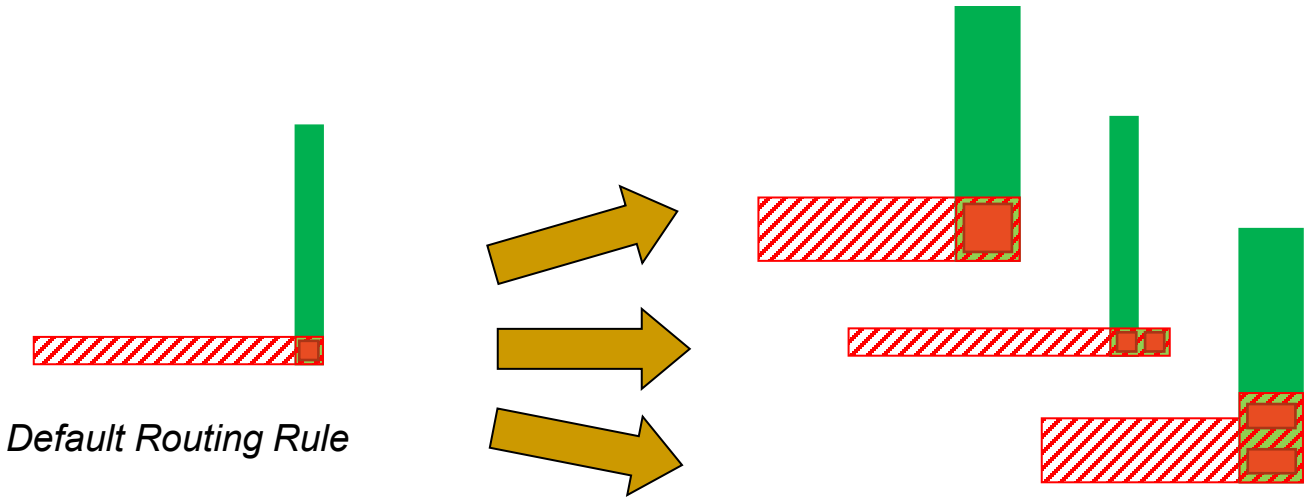
Non-Default Routing Rules

- ***ICC II*** **can route the clock nets using** ***non-default routing*** **(NDR) rules, e.g. double-spacing, double-width, shielding**
  - NDR rules are treated as physical DRCs – reported as violations if not met
- **NDR rules are often used to "harden" the clock, e.g. to make the clock routes less sensitive to** ***cross-talk*** **and** ***electro-migration*** **(*EM*) effects**



Non-Default Via Rules

- **Clock nets are hardened further by often requiring 100% via optimization to enhance reliability**
  - Minimum size single cut vias are replaced with:
    - Multiple-cut via arrays
      and/or
    - Larger “square” or “bar” cuts
- **This is also defined using clock *NDR* rules**



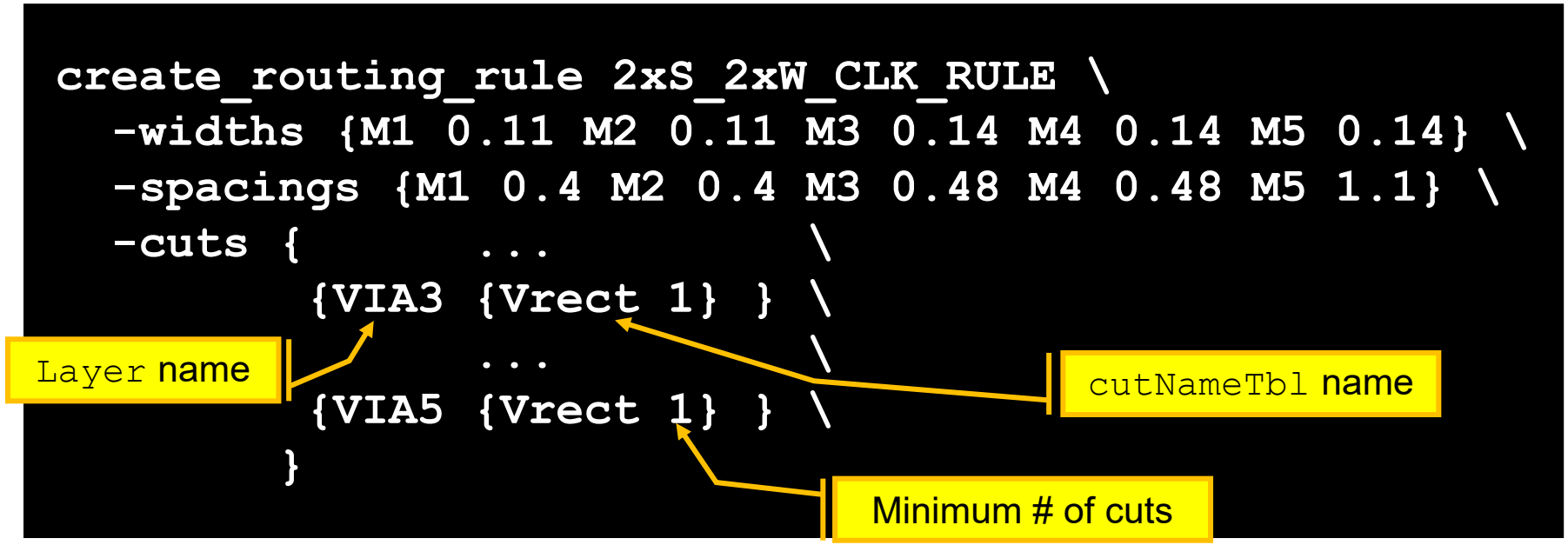
Default Routing Rule

Effect of different via NDRs on Clk routes

Defining Non-Default Routing and Via Rules

- Define the *NDR* rules:

```
create_routing_rule 2xS_2xW_CLK_RULE \
  -widths {M1 0.11 M2 0.11 M3 0.14 M4 0.14 M5 0.14} \
  -spacings {M1 0.4 M2 0.4 M3 0.48 M4 0.48 M5 1.1} \
  -cuts {       ...           \
         {VIA3 {Vrect 1} } \
               ...           \
         {VIA5 {Vrect 1} } \
        }
```

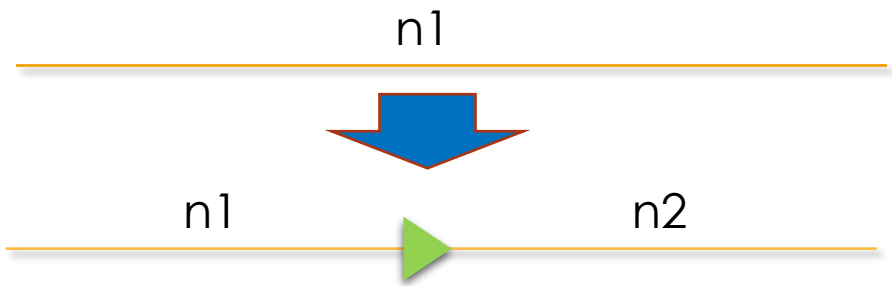


- With the `-cuts` option you use "symbolic" via names defined in the technology file as `cutNameTbl`
- This simplifies the *NDR* definition because one symbolic via can match many specific vias and arrays

NDRs on Clock Nets

- ***ICC II* CTS supports various types of NDR specifications when building the clock tree**
  1. Net-specific NDR from **set_routing_rule <net_list>**:
     NDR will not get propagated to newly created nets
  2. Net-specific NDR from **set_clock_routing_rules -nets**:
     NDR will get propagated to newly created nets
  3. Clock-specific NDR from **set_clock_routing_rules -clocks**:
     Supports separate NDRs for root, sink and internal nets which are clock-specific
  4. Global NDR from **set_clock_routing_rules**:
     Supports separate NDRs for root, sink and internal nets

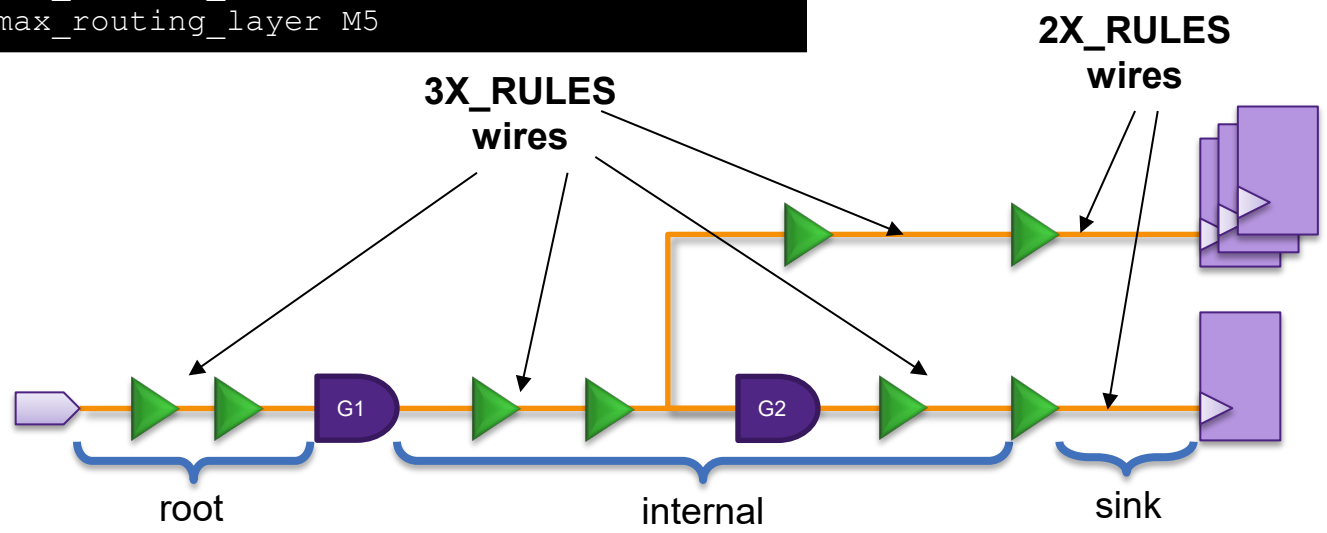
- **The order of priority is 1 > 2 > 3 > 4**



Different NDRs for Root, Internal, Sink Nets

```
set_clock_routing_rules
  -rules 3X_RULES
    -min_routing_layer M4
    -max_routing_layer M5
set_clock_routing_rules
  -net_type sink
  -rules 2X_RULES
    -min_routing_layer M4
    -max_routing_layer M5
```

-net_type:
root | internal | sink

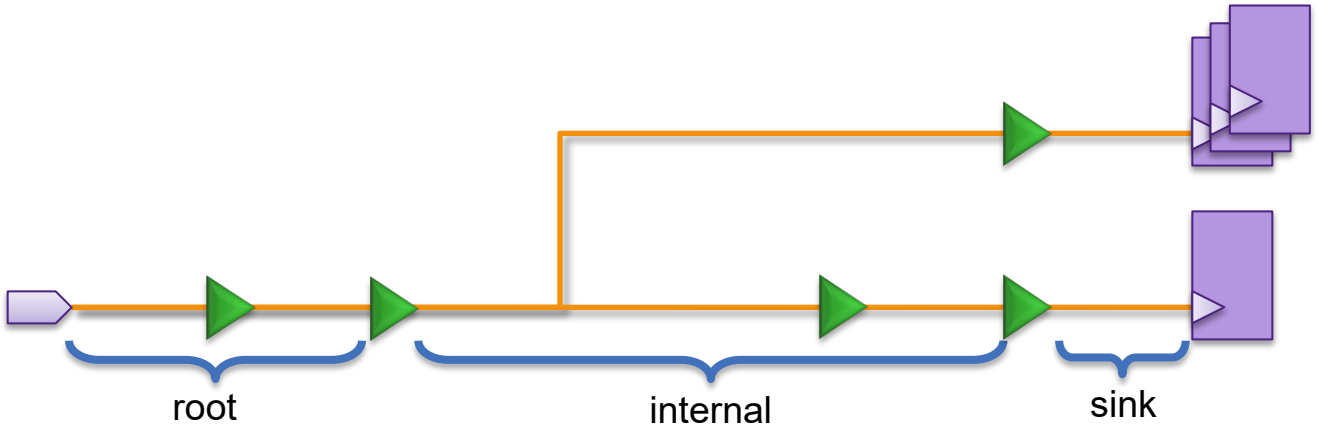


The highlighted options allows you to specify less aggressive NDR rules for clock tree "leaf" nets, while allowing more aggressive NDRs for the other nets.

Helps to prevent routing DRC violations as well as SI issues

Classification of Root, Internal, Sink

- If no gates are present on the clock tree, the first *branching point* of the clock starts the “internal” segment, while the last net from the sink to the buffer is the “sink” segment

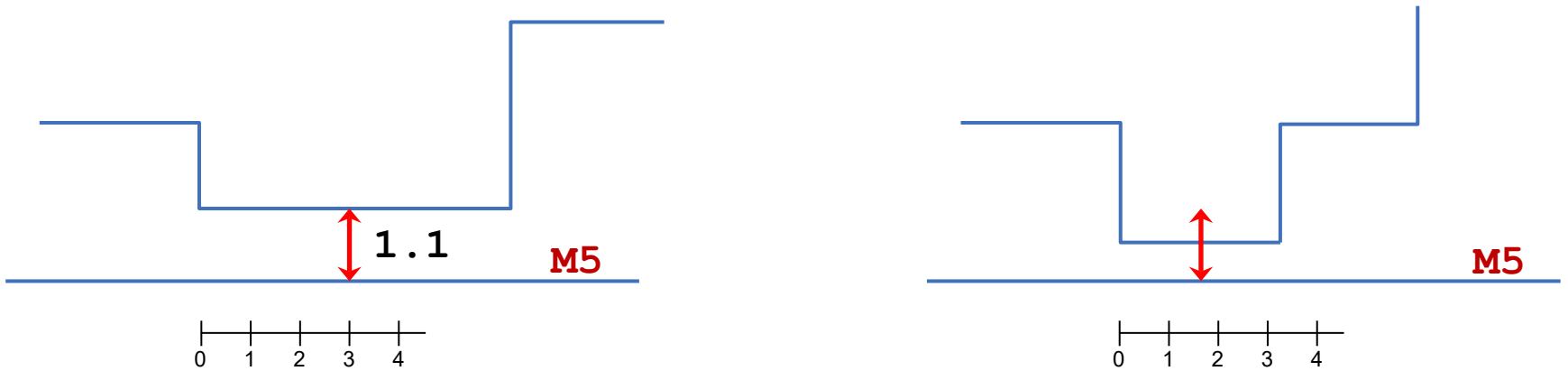


Relaxing Spacing Requirements

- **Increased spacing for crosstalk prevention can make routing difficult**
- **Crosstalk is less of an issue on shorter parallel segments**
  - Relax the NDR by introducing spacing-length thresholds

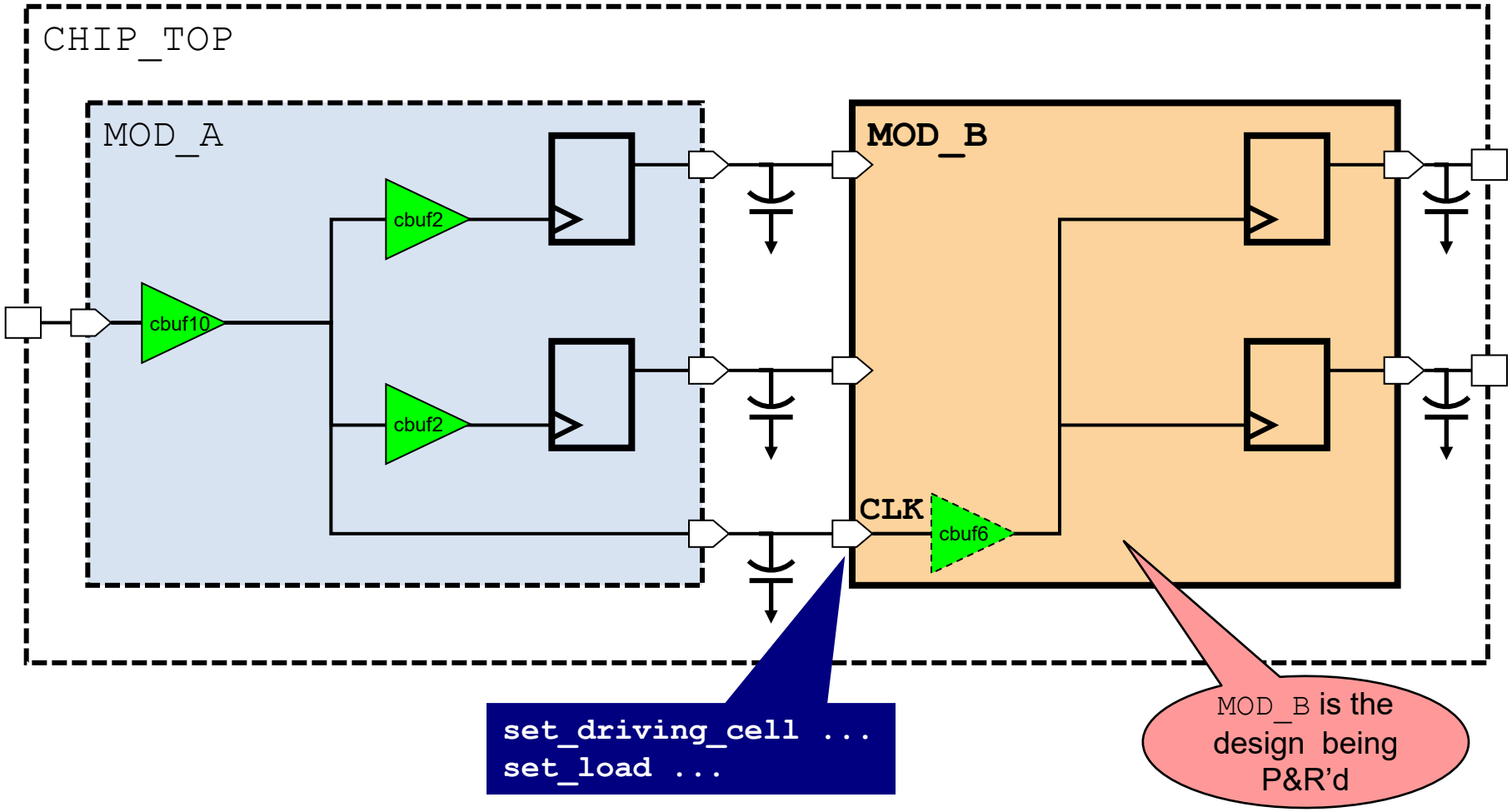
```
create_routing_rule CLK_RULE  \
    -spacings                  { M2 0.16 M3 0.45 M4 0.45 M5 1.1 } \
    -spacing_length_thresholds { M2 3.0  M3 3.0  M4 4.0  M5 4.0 } ...
```

  - Spacing value is ignored if the parallel length is less than the threshold



Are all Clock Drivers and Loads Specified?

Ensure that all clock input ports have a slew constraint –
needed for accurate clock delay calculation during and after CTS



Example: Reporting Clock Port Constraints

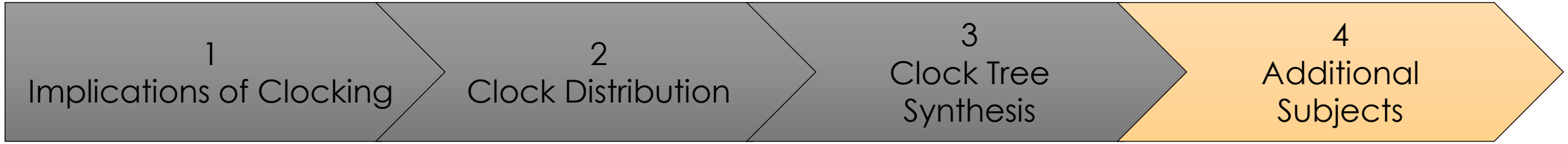
```
report_port -verbose [get_ports *clk]

...
                        Pin Cap                    Wire Cap
                    Min          Max           Min          Max
Port        Dir     rise/fall    rise/fall     rise/fall    rise/fall    Attributes
-------------------------------------------------------------------------------
ate_clk     in      0.00/0.00    0.00/0.00     0.00/0.00    0.00/0.00
pclk        in      0.00/0.00    0.00/0.00     0.00/0.00    0.00/0.00
sdram_clk   in      0.00/0.00    0.00/0.00     0.00/0.00    0.00/0.00
sys_2x_clk  in      0.37/0.37    0.48/0.48     0.00/0.00    0.00/0.00

...

Transition (min/max)
Input Port  Rise              Fall              Related Clock
-------------------------------------------------------------------------------
ate_clk     0.30/0.30         0.30/0.30
pclk        0.12/0.12         0.12/0.12

                    Driving Cell
Input Port      Type      Cell                 Mult      Clock     Attrs
-------------------------------------------------------------------------------
sdram_clk       min_rise  NBUFFX16_RVT
sdram_clk       min_fall  NBUFFX16_RVT
sdram_clk       max_rise  NBUFFX16_RVT
sdram_clk       max_fall  NBUFFX16_RVT
sys_2x_clk      min_rise  NBUFFX16_RVT
sys_2x_clk      min_fall  NBUFFX16_RVT
sys_2x_clk      max_rise  NBUFFX16_RVT
sys_2x_clk      max_fall  NBUFFX16_RVT
```

set_load

set_input_transition

set_driving_cell

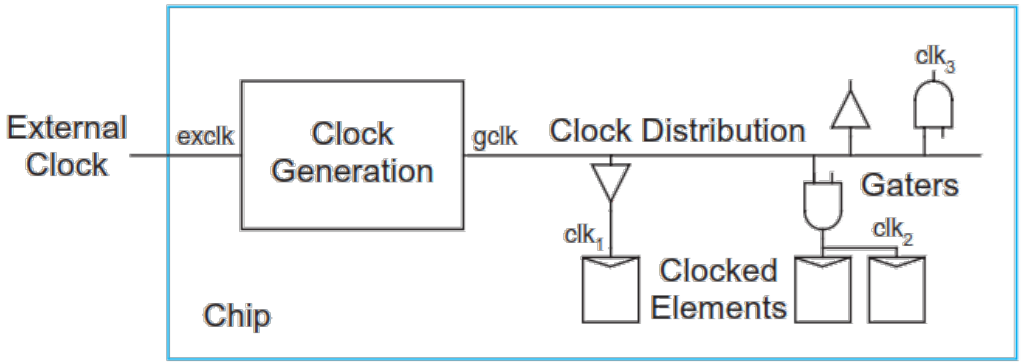
- **Every clock input port should have a `set_driving_cell` or a `set_input_transition` constraint**
  - Models input slew, for accurate timing analysis of the first tiers of clock tree gates
- **Ports with a *driving cell* may additionally require a capacitive load for more accurate slew calculation**
  - Models the external load seen by the driving cell

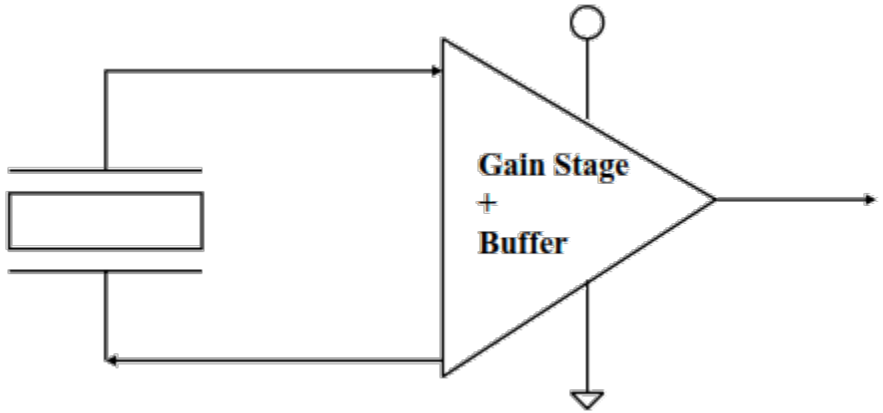


Additional Subjects:
Clock Generation

Clock Generation

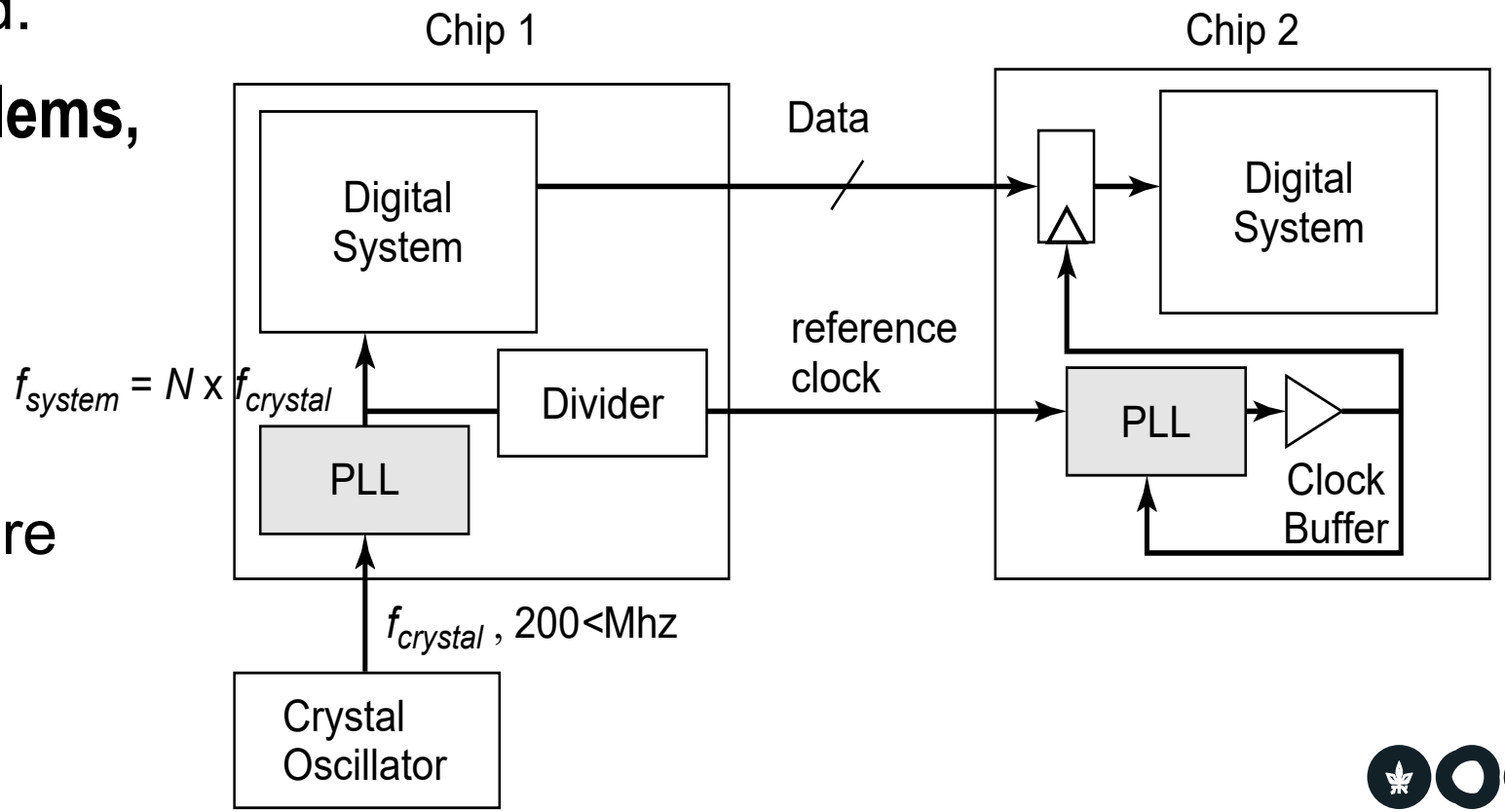
- **Where does the clock come from?**
  - The easiest way to generate a clock is by using a ring oscillator or some other non-stable circuit, but these are susceptible to PVT variations.
  - Therefore, clocks are generally generated off-chip using a crystal and an oscillation circuit.
  - However, usually only one off-chip clock can be used (a single frequency) and the frequency that can be input to the chip is limited to around 100 MHz.
  - Therefore, on-chip local clock generation is employed, usually with a PLL or DLL.





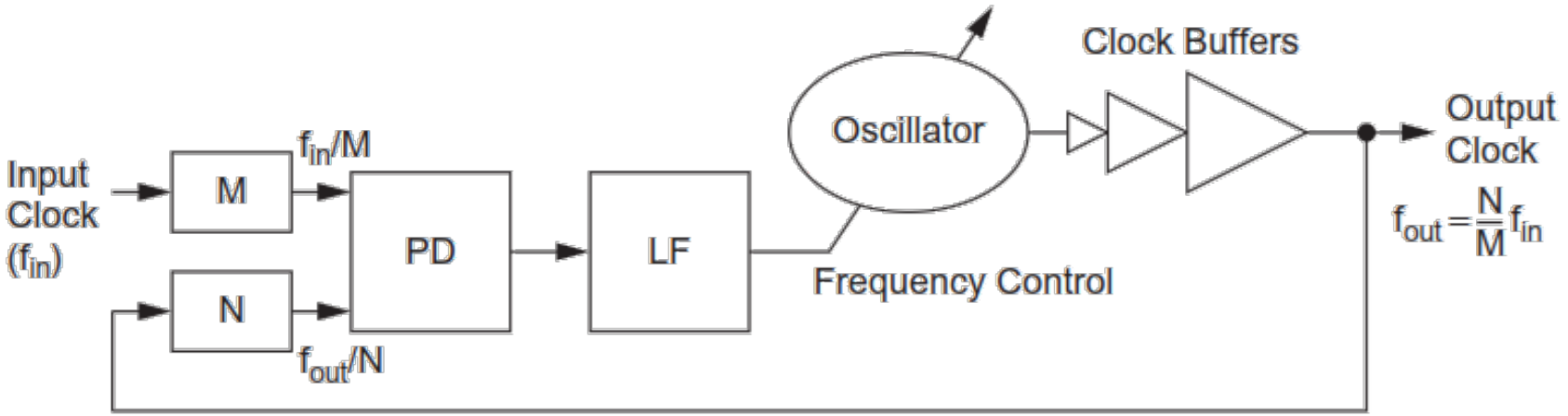
Local Clock Generation

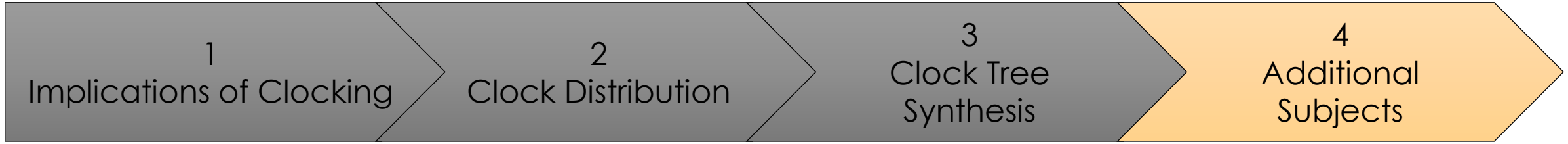
- **Externally generated clocks suffer from two primary problems:**
  - Frequency is limited, i.e., a *clock multiplier* is needed.
  - Clock phase is uncontrolled, such that communication with the external clock domain is unsynchronized.
- **To solve both of these problems, a Phase-Locked Loop (PLL) is used.**
  - If clock multiplication is not required, a delay-locked loop (DLL) is a more simple solution.



Local Clock Generation

- **How does a PLL work?**
  - A *phase detector* (PD) produces a signal proportional to the phase difference between the input and output clocks.
  - A *loop filter* (LF) converts the phase error into a control signal (voltage).
  - A *voltage-controlled oscillator* (VCO), creates a new clock signal based on the error signal.
- **What about a DLL?**
  - Same principle, but instead of changing the frequency, it just delays the clock until the phase is equal.

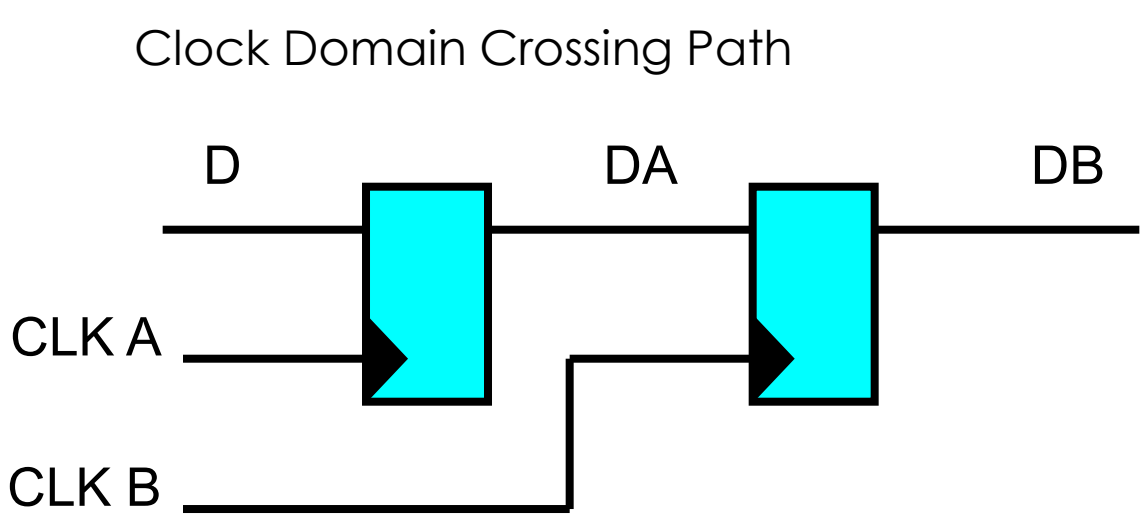


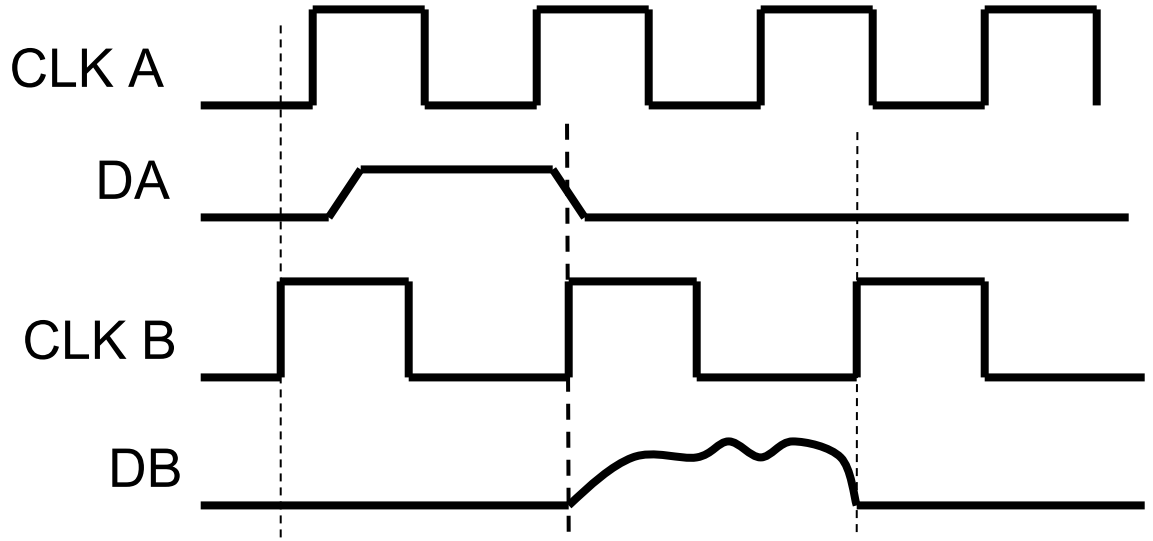


Additional Subjects:
Clock Domain Crossing

Clock Domain Crossing (CDC)

- **Most system-on-chips have several components that communicate with different external interfaces and run on different clock frequencies.**
  - In other words, they have multiple *asynchronous clock domains*.
- **Asynchronous clocks cannot communicate with each other in a straightforward fashion:**





Problems with CDC

The main problems with passing data between asynchronous domains are:

- **Metastability:**
  - A setup/hold violation in the capture register. May cause:
    - High propagation delay at the fanout.
    - High current flow in the chip (even burnout).
    - Different values of the signal at different parts of the fanout.
- **Data Loss:**
  - New data in the source may be generated w/o the data being captured by the destination.
- **Data Incoherency:**
  - Data may be capture late, causing several coherent signals to be in different states.

What is the probability of metastability?

Let us define:

T_w – an error window (setup+hold) around the clock cycle

f – clock frequency

f_D – Frequency of data change

Now assume that a data (D) change can come anywhere in the clock cycle relative to the capture clock, so:

$$Rate(\text{meta}) = f \cdot f_D \cdot T_w$$

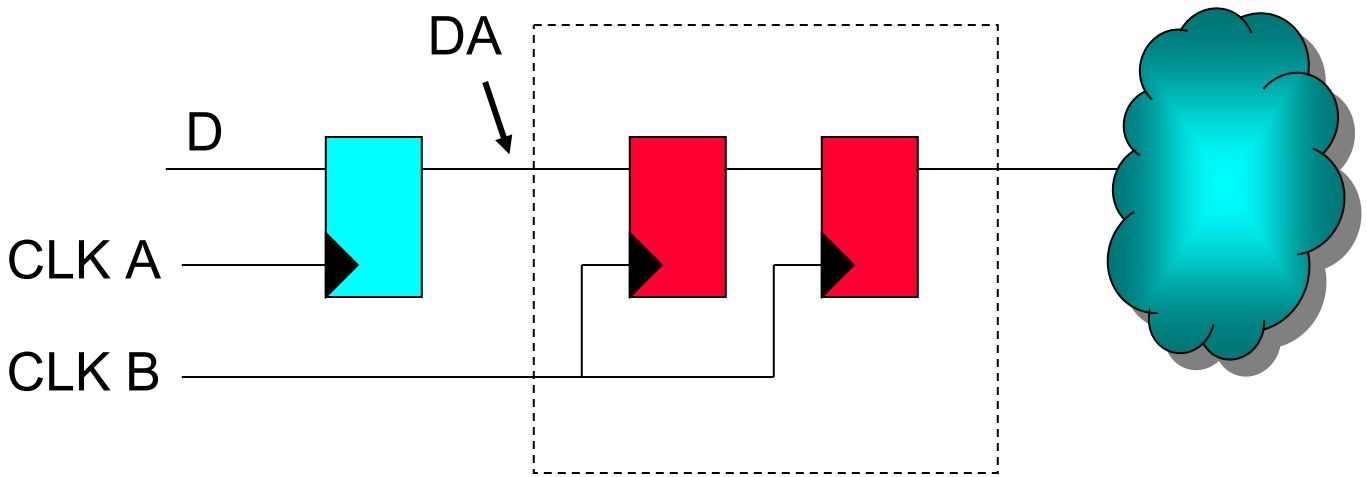
Assume:

$$f = 1\text{GHz} \quad f_D = 0.1f \quad T_w = 20\text{ps}$$

$$Rate(\text{meta}) = 2 \cdot 10^6 / \text{sec}$$

Solutions: Synchronizers

- **By cascading two or more flip flops, we create a simple *synchronizer*.**
  - The signal has one (or more) clock cycles to stabilize.
  - However, there is a probability that the signal will not settle within the cycle time (T)



What is the probability of failure?

The probability for metastability to pass is:

$$P(t > S) = e^{-S/\tau}$$

with τ a parameter of the flip flop

We want the metastability to dissipate at least t_{setup} before the next clock edge, so:

$$P(\text{failure}) = P(\text{meta}) \cdot P(\text{exit}) = \frac{T_w}{T} \cdot e^{-\frac{T - t_{setup}}{\tau}}$$

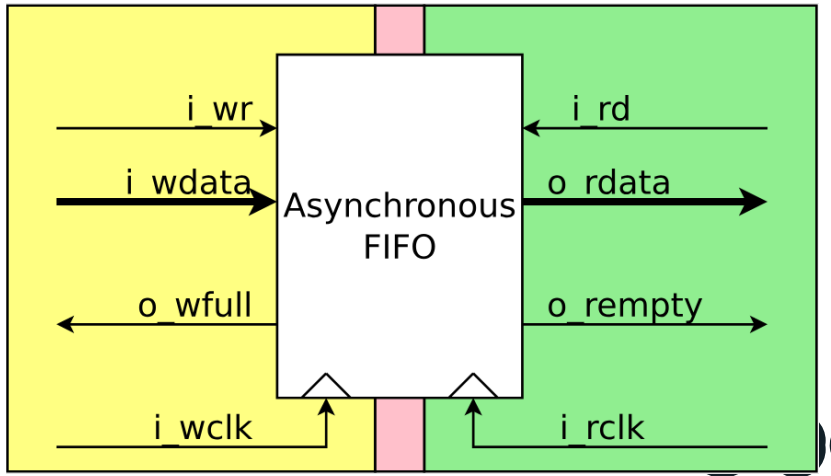
And the mean time between failures (MTBF) is the inverse of the failure rate:

$$MTBF = \frac{1}{Rate(\text{failure})} = \frac{1}{f \cdot f_D \cdot T_w} e^{-\frac{T - t_{setup}}{\tau}}$$

For our previous example, this is about 10^{24} years…

Are synchronizers enough?

- **No!**
  - We may have taken care of metastability,
    but data loss and data incoherence are still there.
  - So we need to design our logic accordingly.
- **To eliminate data loss:**
  - Slow to fast clock – we won't lose any data.
  - Fast to slow clock – hold source data for several cycles.
- **But for data coherence, we need more thinking:**
  - Handshake protocols.
  - First-in First-out (FIFO) interfaces
  - Other solutions (Gray code, Multiplexers, etc.)



Source: ZipCPU.com

Main References

- Berkeley EE141
- Rabaey “Digital Integrated Circuits”
- Synopsys University Courseware
- IDESA
- Gil Rahav
- Dennis Sylvester, UMICH
- MIT 6.375 Complex Digital Systems
- Horowitz, Stanford
- Ginosar, “Metastability and Synchronizers: A Tutorial”